BILKUL SHURUAAT SE · TASVEERON MEIN · KOI PEHLE KA GYAAN NAHI CHAHIYE

Voice AI: zero se, andar tak.

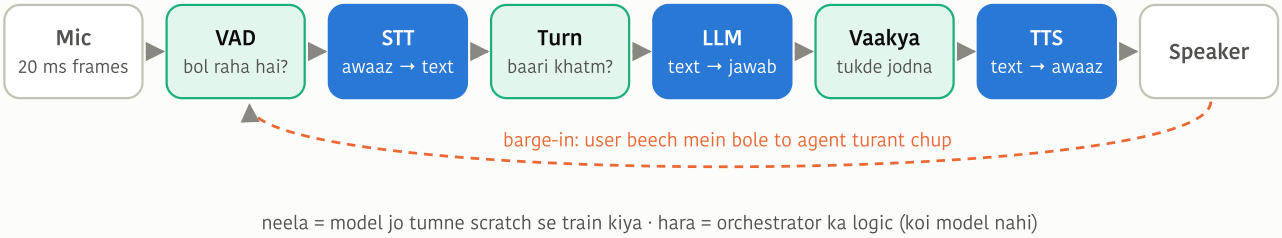
Ek voice agent jo sunta hai, sochta hai aur bolta hai. Uske chaaron hisse (sunna, sochna, bolna, baat-cheet sambhalna) is project mein scratch se bane hain. Ye guide har hisse ko pehle tasveer mein dikhati hai, phir do line mein samjhati hai.

4	24 M	3 + 1	410
hisse: STT, LLM, TTS, orchestrator	parameters, chaaron models milake	Hindi, Marathi, English, aur Hinglish	terms, har ek code ki line se juda

Poora project ek tasveer mein

Tum bolte ho. Machine sunti hai, samajhti hai, jawab sochti hai aur bol deti hai. Beech mein ye aath dabbe hain.

Awaaz andar, awaaz bahar



neela = model jo tumne scratch se train kiya · hara = orchestrator ka logic (koi model nahi)

4.6 M
STT: awaaz ko text banata hai

12.2 M
LLM: jawab likhta hai

4.7 M
TTS: text ki tasveer (mel) banata hai

2.8 M
Vocoder: tasveer ko awaaz banata hai

PARAMETER KYA HAI?

Model ke andar ka ek knob (ek number). Training in knobs ko ghumati hai jab tak jawab sahi na aane lage. ChatGPT mein ye kharabon mein hain; hamare chaaron models milake 24 M. Chhota hai, isliye laptop pe ban gaya aur tum har line padh sakte ho.

Is guide ko kaise padhna hai

KAHANI

Rozmarra ki misaal. Pehle ye samjho, phir asli naam.

SEEDHI BAAT

Wahi cheez do-teen line mein.

INTERVIEW MEIN BOLO

Wahi baat English mein, jaise bolni hai.

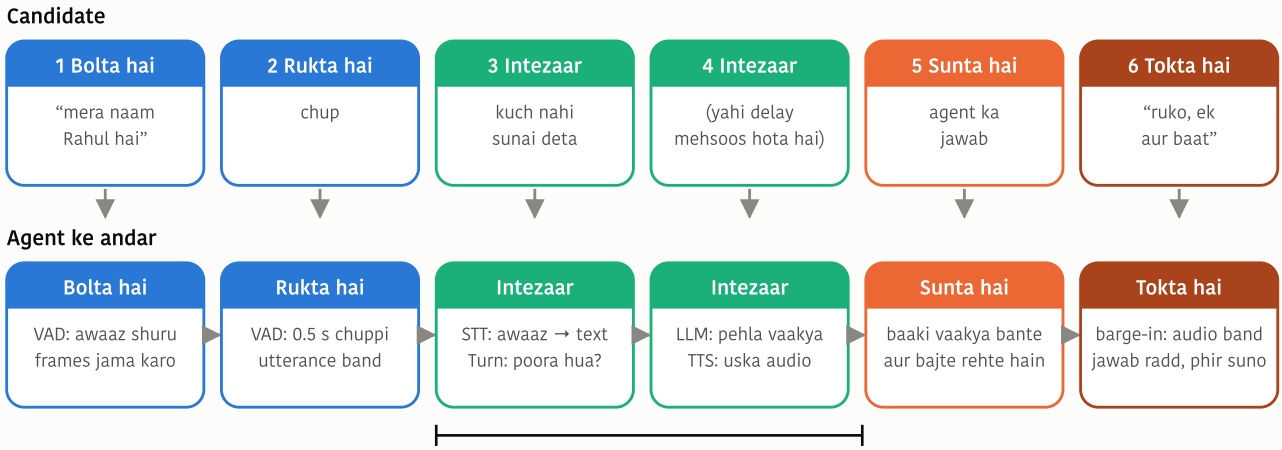
SACHCHI BAAT

Jahan model kamzor hai ya kuch nahi bana, wahan saaf likha hai.

Candidate ka safar: ek baari, shuru se ant tak

Upar: insaan kya karta aur mehsoos karta hai. Neeche: usi pal machine ke andar kya chal raha hai.

Ek sawaal, ek jawab, ek tokna



time to first audio: candidate ruka → agent bola. Poora project isi ko chhota karne ke baare mein hai.

Kadam 1-2

Mic hamesha khula hai. VAD pehchanta hai ki bolna kab shuru hua aur kab ruka.

Kadam 3

STT awaaz ko text banata hai. Turn detector poochta hai: baat poori hui ya beech ka rukna tha?

Kadam 4-5

LLM jawab likhna shuru karta hai. Pehla vaakya bante hi TTS use bol deta hai; baaki peeche banta rehta hai.

Kadam 6

Candidate beech mein bola to agent usi pal chup. Ise barge-in kehte hain.

ACCHA EXPERIENCE KYA HAI?

Teen cheezein: agent jaldi jawab de (kam delay), beech mein na toke (false cutoff na ho), aur tokne pe turant ruk jaaye. Teeno ek doosre se ladti hain: jaldi jawab doge to tokne ka khatra badhta hai. Chapter 7 isi santulan ke baare mein hai.

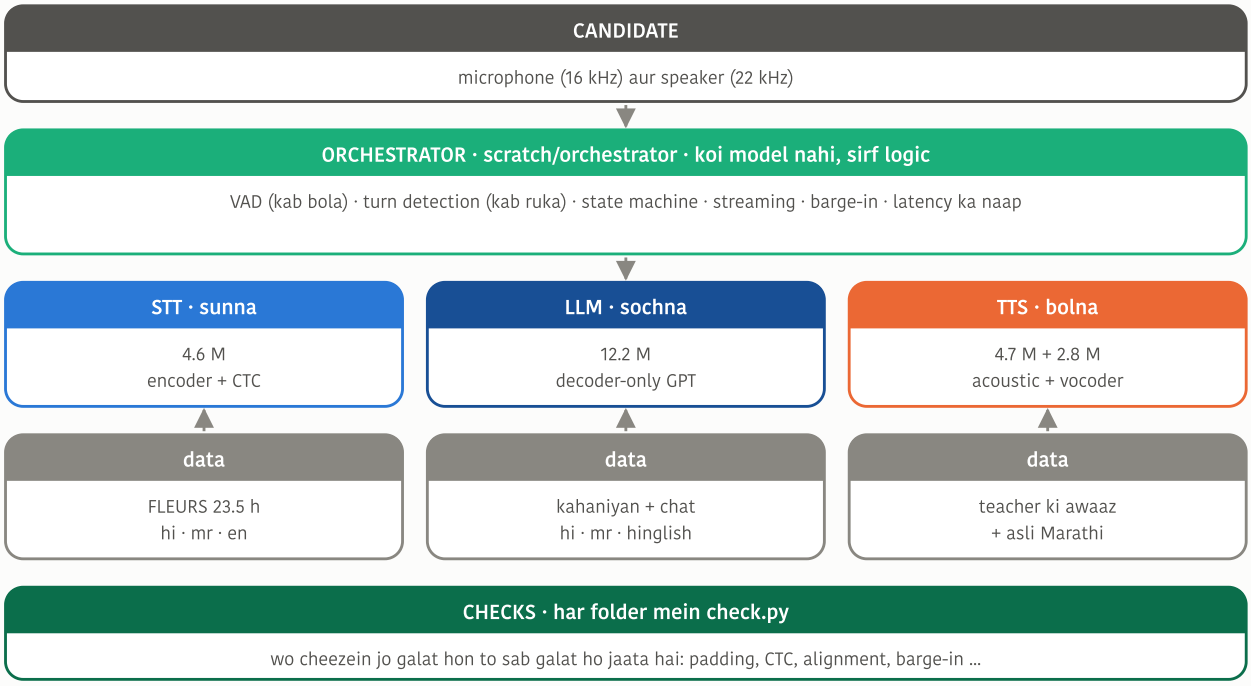
INTERVIEW MEIN BOLO

"It is a cascaded voice pipeline: VAD, speech-to-text, a turn detector, an LLM that streams tokens, sentence chunking, and text-to-speech. The metric I optimise is time to first audio, and the agent supports barge-in."

Architecture: paanch parton mein

Upar insaan, neeche data. Har part sirf apne upar-neeche wale se baat karta hai.

Kaun kiske upar khada hai



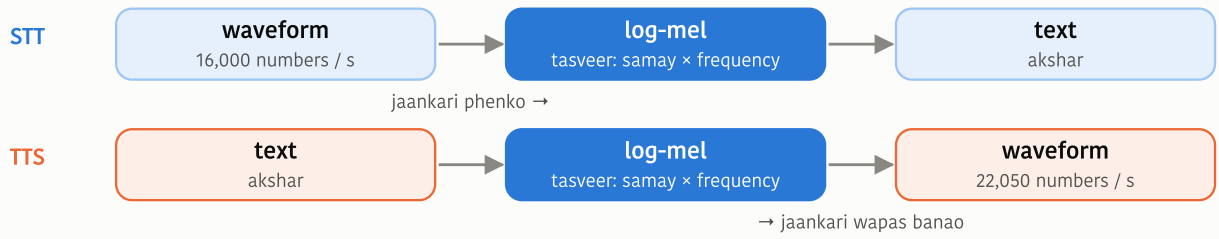
"SCRATCH SE" KA MATLAB

Model ka har hissa (attention, CTC ka loss, alignment, vocoder), training loop aur orchestration is folder mein haath se likha hai. PyTorch sirf tensors aur gradients ke liye hai. Koi bana-banaya model load nahi hota.

SACHCHI BAAT

Data bahar se aaya hai (FLEURS, OpenSLR), aur TTS ne ek bade model ki awaaz ki nakal karke seekha hai (distillation). Ye model asli agent ke models ki jagah nahi lenge; inka kaam samjhana hai.

Sunna aur bolna ek doosre ka aaina hain



Beech ki cheez dono mein wahi hai: log-mel spectrogram. STT use padhta hai, TTS use banata hai.

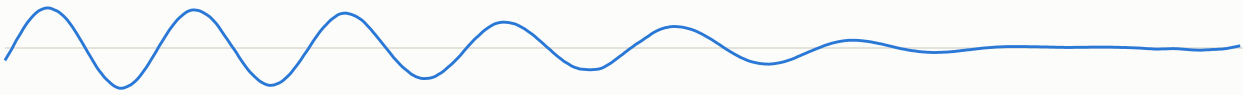
1 Awaaz numbers kaise banti hai

dekho: `stt/audio.py` · lesson 1

KAHANI

Talaab mein pathar phenko to lehrein uthti hain. Awaaz bhi hawa ki lehar hai. Mic ek aisa chowkidaar hai jo har second 16,000 baar likh leta hai ki lehar abhi kitni oonchi hai. Wahi lambi list "awaaz" hai.

Asli awaaz ke 25 millisecond



25 ms ki awaaz = 551 numbers (22,050 samples har second). Yahi ek 'frame' hai.

Sample

List ka ek number: ek pal ki oonchai.

Sampling rate

Ek second mein kitne numbers. Sunne ke liye 16,000; bolne ke liye 22,050.

Frame

25 ms ka tukda. Itni der awaaz "ek jaisi" maani jaati hai.

Hop

Har 10 ms pe naya frame. Yaani 100 frames har second.

DIKKAT

In numbers mein "a" ya "k" kahin nahi dikhta. Swar ko swar banati hain uski **frequencies**: kaunse sur zor se baj rahe hain. Wo is list mein chhupi hain. Isliye tasveer badalni padti hai.

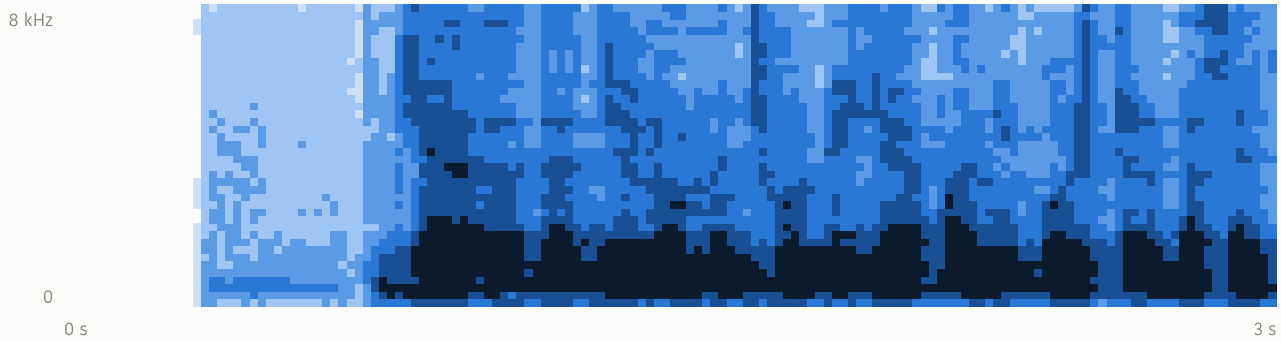
INTERVIEW MEIN BOLO

"Audio is a sequence of samples: 16,000 per second for recognition. I parse the WAV header myself, convert to mono floats, and work on 25 millisecond frames with a 10 millisecond hop."

2 Spectrogram: awaaz ki tasveer

dekho: `stt/audio.py` · `power_spectrogram`, `mel_filterbank`, `log_mel`

Samay baayein se daayein, frequency neeche se upar, gehra rang = zyada zor



Asli Hindi clip ka log-mel (dev split): “ज़्यादातर जिलों में छोटी जापानी कोस्टर बसें चल …”

1. Frames

Awaaz ko 25 ms ke tukdon mein kaato.

2. Fourier

Har tukde se poocho: kaunsi frequency kitni hai?

3. Mel

201 frequencies ko 80 pattiyon mein sameto, kaan ki tarah (neeche baareek, upar moti).

4. Log

Kaan anupaat sunta hai, isliye log lo.

INTERVIEW MEIN BOLO

"I convert the waveform to an 80-band log-mel spectrogram: 25 millisecond Hann windows every 10 milliseconds, FFT, a mel filterbank, then log. I wrote the STFT by hand and tested it against torch.stft."

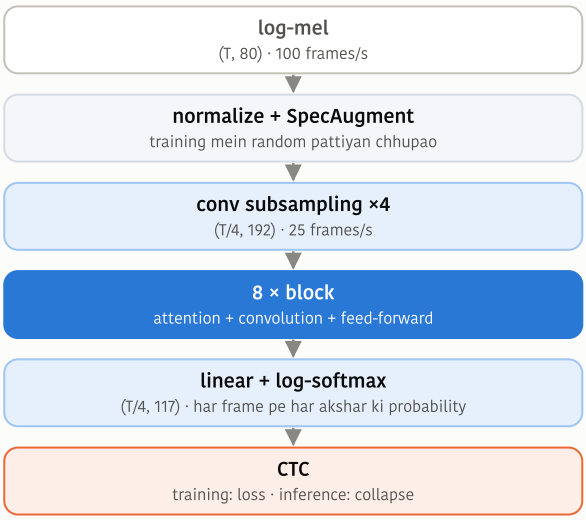
3 STT: tasveer se akshar

dekho: `stt/model.py`, `stt/ctc.py` · lessons 4-5

Kaam ek line mein

Har 40 millisecond ke liye batao: is pal kaunsa akshar bola ja raha hai, ya koi nahi.

Model: 4.6 M parameters



KAHANI

Ek clerk film ki har 4 frame mein se 1 dekhta hai (subsampling). Phir har frame ke liye do kaam: aas-paas ke frames dekhna (convolution: "ye 'k' ki shakal hai") aur poori film mein kahin bhi jhaankna (attention: "aage 'kal' aa raha hai, to ye 'k' hi hoga").

ENCODER-ONLY

LLM ek-ek token likhta tha aur peeche hi dekh sakta tha. Ye model poora clip ek saath dekhta hai, aage-peeche dono, aur saare frames ek hi baar mein nikalta hai. Isliye tez hai.

PADDING KA JAAL

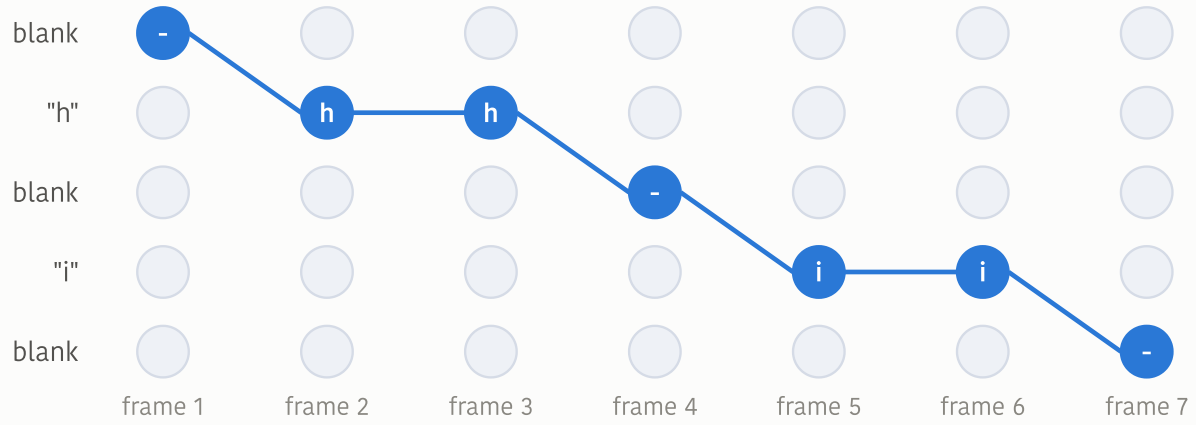
Batch mein chhote clips ke peeche khaali jagah bharni padti hai. Agar model us khaali jagah ko "dekh" le to wahi clip akele aur batch mein alag jawab dega. `check.py` yahi pakadta hai.

CTC: jab koi nahi batata ki kaunsa akshar kab bola gaya

KAHANI

5 second ki awaaz = 125 frames. Text = 40 akshar. Kaunsa frame kis akshar ka? Kisi ne nahi likha. CTC kehta hai: mat batao. Model har frame pe ek akshar ya "blank" bole; phir do niyam lagao: **1.** lagataar dohraye akshar ek kar do, **2.** blank hata do.

"hi" ka ek raasta, 7 frames mein



ye raasta: - h h - i i - → repeats milao → - h - i - → blank hatao → "hi"

`h h – e – l l – l – o → h – e – l – l – o → hello`

Do "l" isliye bache kyunki unke beech ek blank tha. Blank na hota to niyam 1 unhe ek kar deta.

HAATH SE HISAAB: 2 FRAMES, HAR FRAME MEIN BLANK 0.6, "A" 0.4	RAASTA	TEXT	PROBABILITY
Sabse sambhav **raasta** "- -" hai (0.36), par sabse sambhav **text** "a" hai (0.64), kyunki teen raaste "a" dete hain. CTC ka loss = –ln(sahi text ke saare raaston ka jod).	- -	""	0.36
	- a	a	0.24
	a -	a	0.24
	a a	a	0.16

INTERVIEW MEIN BOLO

"CTC removes the need for frame-level labels. The model emits a character or a blank per frame, and the loss sums the probability of every path that collapses to the transcript, using the forward algorithm. I implemented it and it matches PyTorch's value and gradient."

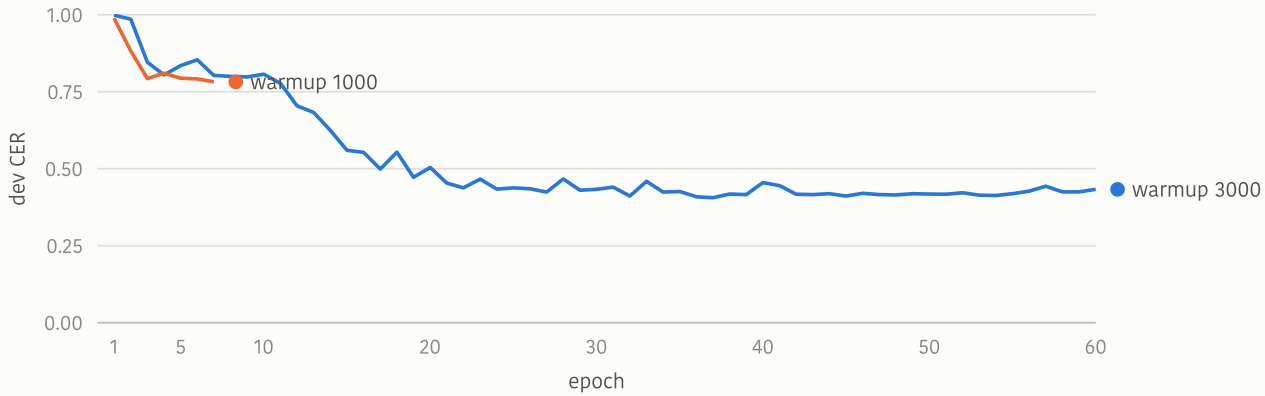
4 STT ki training: jo hua, sach-sach

dekho: `stt/train.py`, `stt/runs/train.log` · lesson 6

23.5 h	7,594	117	0.000
training audio (FLEURS)	clips: Hindi, Marathi, English	akshar ki vocabulary (blank ke saath)	CER, jab 16 clips ratwaye (overfit test)

Overfit test pehle, hamesha: poori training se pehle model ko sirf 16 clips ratwao. Rat le to pipeline sahi judi hai; na rate to code mein bug hai, aur koi data use theek nahi karega.

Dev CER har epoch ke baad (neeche = behtar; 1.00 = kuch sahi nahi)



CER = character error rate: 100 aksharon mein kitne galat. **Dev** = wo clips jo model ne training mein kabhi nahi dekhe.

JO GALAT GAYA

Pehli run mein learning rate bahut jaldi poora kar diya (1000 steps mein). Model ne sabse sasta dhokha seekh liya: "hamesha blank bolo". Loss atak gaya, CER 0.78 pe. Ise **blank plateau** kehte hain.

JO SAHI KIYA

Ek chhota experiment chalaya jisme learning rate dheere badha: 5 epoch mein CER 0.57. Saboot milne ke baad warmup 3000 steps kiya aur training dobara shuru ki. Neeli line wahi hai.

Training 60 epochs chali. Sabse accha dev CER **0.406** epoch 37 pe aaya; uske baad train loss girta raha aur dev CER badha (overfitting), isliye wahi checkpoint rakha.

Test clips pe (jo model ne kabhi nahi dekhe), ek 300 M model ke saath

BHASHA	HAMARA CER	HAMARA WER	300 M MODEL CER	300 M MODEL WER
Hindi	0.323	0.747	0.095	0.276
Marathi	0.402	1.091	0.115	0.453
English	0.446	0.923	0.061	0.178

Real-time factor (CPU, 2 threads; kam = tez): Hindi: hamara 0.023, bada 0.238 · Marathi: hamara 0.029, bada 0.312 · English: hamara 0.014, bada 0.158

ZYADA DATA, WAHI MODEL

Wahi 4.6 M model, 23 ghante ki jagah 158 ghante (+ Kathbath Hindi), muft Kaggle T4 pe 1.3 ghante. Test CER: Hindi **0.323 → 0.110**, Marathi **0.402 → 0.193**, English 0.446 → 0.369. Kathbath sirf Hindi hai, phir bhi Marathi aadha hua: dono Devanagari aur milte-julte sound.

en asli: he built a wifi door bell he said
en model: tyo b u a wiye ait gw aro eloi hic ayid

hi asli: पुलिस ने कहा कि शव लगभग एक दिन पुराना है
hi model: पुलिस्िका कि शब लगखक कएकनेन पुरणा है

mr asli: वैज्ञानिकांच्या सांगण्याप्रमाणे धडकेमुळे झालेला स्फोट प्रचंड होता
mr model: बजने गे चेा साना प्रमीं जडीकेउमनी जारेला स पूट कूजन होता

INTERVIEW MEIN BOLO

"My first run sat on the CTC blank plateau. I ran a small controlled experiment, found the warmup was too short, fixed it, and kept both logs. I always run an overfit test on 16 clips before a full run."

5 LLM: jawab sochna

dekho: `llm/` · iski apni do guides hain

Kaam ek line mein

Ab tak ke shabd dekh ke agla shabd batao. Baar-baar. Wahi jawab hai.

KAHANI

Phone ka keyboard jo agla shabd sujhata hai, bas bahut behtar. 12.2 M knobs, Hindi, Marathi aur Hinglish ki kahaniyon aur chat pe train hua.

VOICE AGENT MEIN ISKE DO KAAM

1. Jawab likhna, token-dar-token, taaki pehla vaakya turant TTS ko mile. **2.** Ye batana ki candidate ki baat poori hui ya nahi (chapter 7).

YE CHEEZ	KAHAN BANI	EK LINE MEIN
Backpropagation	`llm/autograd.py`	Galti ko peeche ki taraf baantna, taaki har knob ko apna hissa pata chale
Tokenizer (BPE)	`llm/tokenizer.py`	Text ko tukdon mein, tukdon ko numbers mein
Attention + transformer	`llm/model.py`	Har shabd peeche ke kaam ke shabdon se jaankari leta hai
Sampling, KV cache	`llm/generate.py`	Probabilities se shabd chunna; purana hisaab yaad rakhna
LoRA, quantization	`llm/lora.py`, `llm/shrink.py`	1% knobs se nayi aadat; model ko 4 guna chhota karna

POORI KAHANI YAHAN HAI

`llm/docs/LLM-Zero-Se-Interview-Tak.pdf` (10 seedhiyan, bilkul shuru se) aur `llm/docs/Chhotu-GPT-Andar-Se.pdf` (34 pages, saare graphs). STT aur TTS ke blocks isi attention, RMSNorm aur RoPE pe bane hain, to LLM wali guide pehle padhna faydemand hai.

SACHCHI BAAT

Ye LLM chhoti Hinglish baat kar leta hai, facts ya lambi soch nahi. Orchestrator mein ek flag se asli agent ka Gemma lag jaata hai (`--llm gemma`); baaki code nahi badalta.

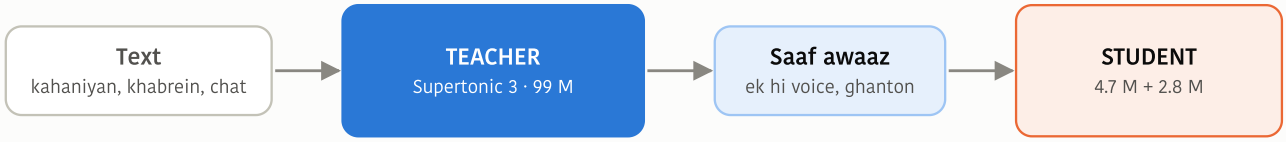
6 TTS: akshar se awaaz

dekho: `tts/` · lessons 1-9

Kaam ek line mein

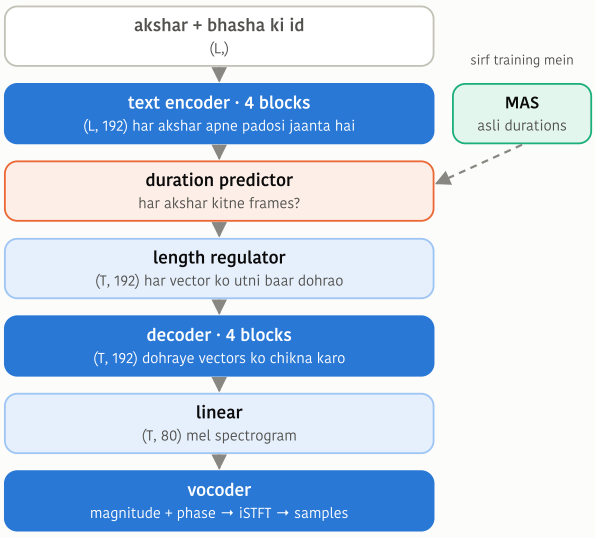
Vaakya padho, tay karo har akshar kitni der bolna hai, uski tasveer (mel) banao, phir tasveer ko awaaz mein badlo.

Data kahan se aaya: bade model ki nakal (distillation)



Student wahi vaakya, wahi awaaz dekhta hai aur nakal karna seekhta hai. Teacher se aage nahi ja sakta, paas pahunch sakta hai.

Model: 4.7 M + vocoder 2.8 M



KAHANI

Gaayak ke saamne bol likhe hain. Pehle wo tay karta hai har akshar kitna kheenchna hai (duration). Phir sur aur awaaz bharta hai (decoder). Aakhir mein gala use asli dhwani banata hai (vocoder).

SAARE FRAMES EK SAATH

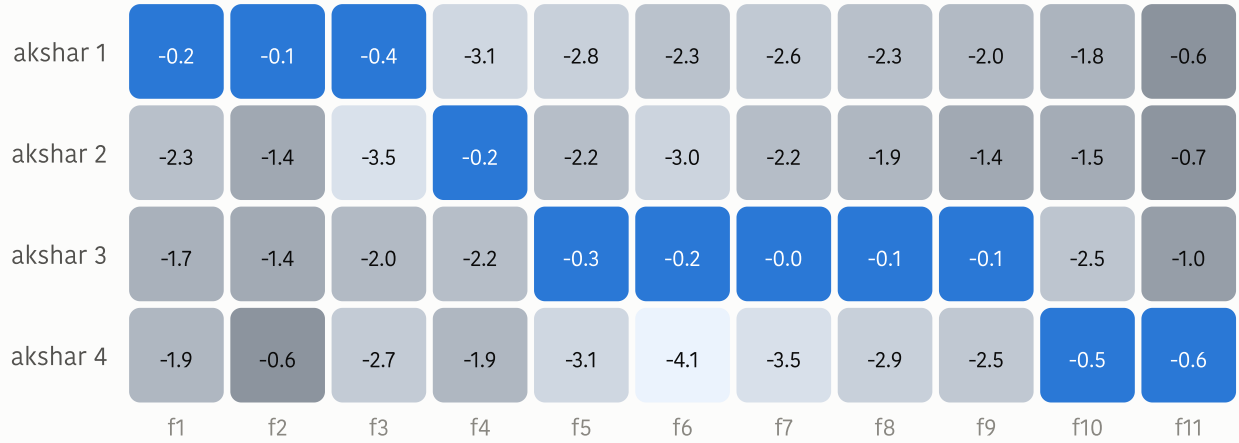
Purane models (Tacotron) ek-ek frame banate the aur kabhi shabd chhod dete ya dohrate the. Yahan durations pehle tay ho jaati hain, to chhootna ya dohrana ho hi nahi sakta, aur sab frames ek hi baar mein bante hain.

PUNCTUATION YAHAN ZAROORI HAI

STT ne comma phenk diya tha kyunki wo sunai nahi deta. TTS use rakhta hai: comma = chhota rukna, sawaaliya nishaan = uthta sur.

Har akshar kitna lamba? MAS khud dhoondhta hai

4 akshar, 11 frames: har khaane mein "ye akshar is frame se kitna milta hai" (0 ke kareeb = accha)



neela = MAS ka raasta. Durations: [3, 1, 5, 2] (asli: [3, 1, 5, 2])

NIYAM

Raasta sirf daayein ya daayein-neeche ja sakta hai: akshar kram mein, koi chhoota nahi. Jis akshar ko jitne khaane mile, wahi uski duration.

VOCODER KA KAMAAL

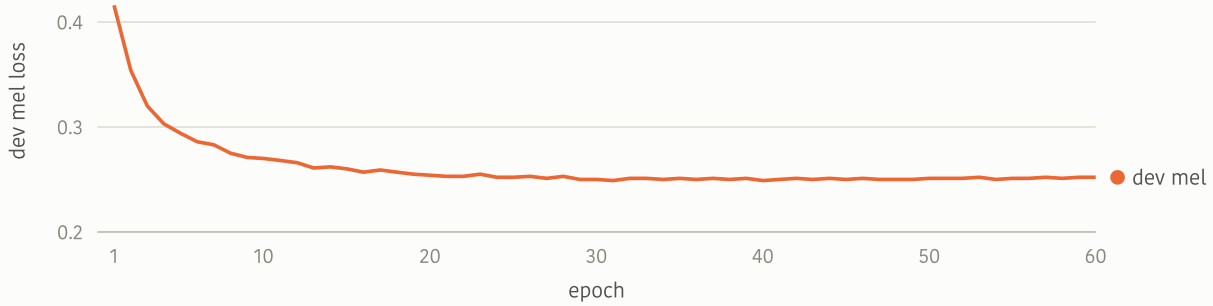
Mel mein "phase" nahi hota (lehar apne chakkar mein kahan hai). Purana tareeka (Griffin-Lim) andaza lagata hai aur jhanjhanata hai. Hamara vocoder har frame ka zor aur phase dono seekhta hai, phir ek ulta-Fourier se awaaz banti hai.

Kitna data, aur naapa kaise

BHASHA	CLIPS	AUDIO
Hindi (teacher)	2,960	3.16 h
English (teacher)	1,073	2.20 h
Hinglish, Roman (teacher)	2,460	3.28 h
Marathi (asli recording)	1,511	2.27 h

Naapne ka tareeka: hamari TTS ki awaaz ek mazboot recognizer (300 M) ko sunao aur dekho usne kitne akshar galat likhe. Wahi vaakya teacher ki awaaz mein bhi sunao. Dono ka fark = student ne kya khoya.

Acoustic model ki training: dev mel loss (kam = asli awaaz ke kareeb)



Sabse accha epoch 31 pe (0.249); uske baad sirf train loss gira. 11 ghante data ki yahi seema hai.

BHASHA	STUDENT CER	TEACHER CER	REAL-TIME FACTOR
Hindi	0.038	0.035	0.034
Marathi (teacher = asli recording)	0.090	0.043	0.027
English	0.035	0.016	0.021
Hinglish, Roman	0.295	0.293	0.029

40 dev vaakya har bhasha, judge Omnilingual ASR 300 M, CPU 2 threads. Hinglish ka 0.29 judge ki galti hai (Roman Hinglish ki fixed spelling nahi): wahan student aur teacher ka fark dekho. Real-time factor 0.03 = real time se ~30 guna tez.

INTERVIEW MEIN BOLO

"It is a non-autoregressive FastSpeech-style model. Durations come from monotonic alignment search during training, a duration predictor replaces it at inference, and a Vocos-style vocoder predicts magnitude and phase for an inverse STFT. The training audio was distilled from a larger teacher model."

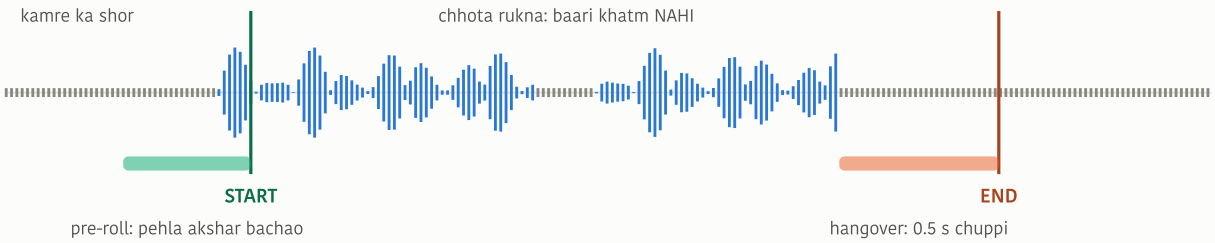
7 Orchestrator: teen models ko baat-cheet banana

dekho: `orchestrator/` · lessons 1-5

Kaam ek line mein

Tay karna ki kaun kab bolega. Isme koi model nahi hai, sirf samay ka hisaab aur ek state machine.

1. VAD: koi bol raha hai?



Har 20 ms ka frame: kamre ke shor se 12 dB zyada zor ho to "awaaz". **START** 100 ms lagataar awaaz ke baad, **END** 500 ms lagataar chuppi ke baad.

2. Turn detection: baat poori hui, ya bas soch raha hai?

KAHANI

"mera order number hai ..." ke baad rukna aur "haan theek hai." ke baad rukna: dono chuppi hain. Pehle pe jawab doge to tumne toka. Doosre pe 2 second rukoge to agent mara hua lagega.

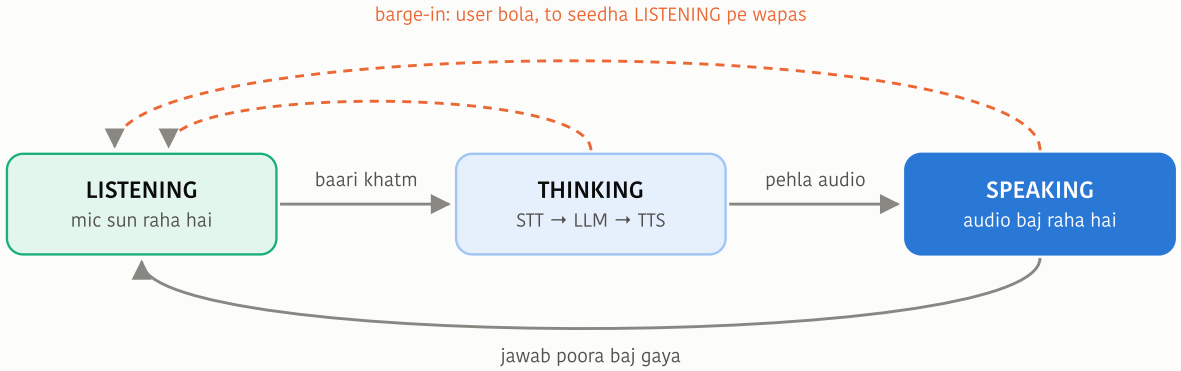
HAMARA TAREEKA

Apne LLM se poochte hain: ab tak ke shabdon ke baad "vaakya khatm" aane ki kitni sambhavna hai? Zyada ho to 0.3 s mein jawab; kam ho to 1.5 s tak sabr.

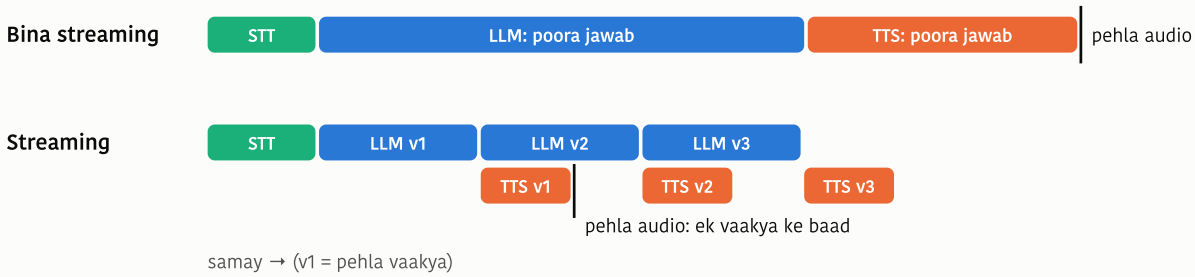
AB TAK JO BOLA	SACH	P(POORA)
mera order number hai	rukna	0.389
kal main office	rukna	0.000
mujhe lagta hai ki	rukna	0.000
agar tum free ho to	rukna	0.016
main soch raha tha ki hum	rukna	0.000
haan theek hai	khatm	0.014
kal main office nahi aaunga	khatm	0.193
tum kahan ho?	khatm	0.984
chalo phir milte hain	khatm	0.120
kya kar rahe ho aaj?	khatm	0.943

Ye numbers is guide ko banate waqt hamare LLM se nikle hain. 10 haath ke likhe udaharan hain: tareeka dikhane ke liye, benchmark nahi.

3. State machine aur barge-in



4. Streaming: poore jawab ka intezaar mat karo



Dono mein kul kaam barabar hai. Fark ye hai ki candidate ko **pehli awaaz** kab sunai deti hai.

RADD KARNA (CANCELLATION)

Har jawab ka ek number hota hai. User toke to number badal jaata hai, aur purane number wala har kaam khud ruk jaata hai. Isse tooti hui reply baad mein bol nahi padti.

SACHCHI BAAT: ECHO

Speaker ki awaaz mic mein wapas jaati hai, to agent khud ko sun ke khud ko tok dega. Phone aur browser ise "echo cancellation" se rokte hain. Is script mein wo nahi hai: headphone lagao.

5. Asli teeno models ke saath, naapa hua

MS	INTEZAAR	STT	LLM	TTS	PEHLI AWAAZ
Ek turn	1350	153	52	123	1776
Barge-in	1396	105	22	62	1612
Barge-in: naya jawab	1467	33	11	78	1640

Ek FLEURS Hindi clip, M1 Pro CPU. Teeno model milke 150 ms se kam lete hain; baaki intezaar turn detection ka hai.

INTERVIEW MEIN BOLO

"The orchestrator is a three-state machine. Model calls run in worker threads so the event loop keeps reading the microphone, which is what makes barge-in possible. Each reply has a generation counter, so a cancelled reply can never speak later. I test it with fake models of known delay, and end to end with the three trained models."

8 Naksha: kaunsi file kya karti hai

ye table code se khud bani hai: har file ki pehli line

Har folder mein files lesson ke order mein hain. Upar ka docstring padho, phir code, phir chalao. Har folder mein ek `check.py` (jaanch) aur ek `GLOSSARY.md` (shabd) bhi hai.

KHUD CHALAO

`cd scratch/stt && ../../.venv/bin/python audio.py` har lesson file aise hi chalti hai aur kuch dikhati hai.

`../../.venv/bin/python check.py` har folder ki jaanch, kuch second mein.

`cd scratch/orchestrator && ../../.venv/bin/python agent.py` teeno models se baat karo (headphone lagake).

LLM: text ka dimaag `scratch/llm/`

FILE	LESSON	KYA KARTI HAI (FILE KI PEHLI LINE SE)	LINES
`autograd.py`	1	autograd: how a neural network learns, in plain Python (no PyTorch, no NumPy)	166
`tokenizer.py`	2	tokenizer: turning text into numbers with byte-pair encoding (BPE), written from scratch	160
`data.py`	3	data: from raw text on the internet to the integer arrays the model trains on	91
`model.py`	4	the model: a decoder-only transformer (a GPT), every layer written out	222
`train.py`	5	training: the loop that turns random weights into a language model	130
`generate.py`	6	generation: turning "a probability for every next token" into text	110
`lora.py`	7	fine-tuning with LoRA: teach a trained model something new by training under 1% of it	126
`shrink.py`	8	quantization: making the model small enough for a phone, and measuring what it costs	59
`xray.py`	9	x-ray: look inside the model while it reads a prompt	60
`check.py`		Checks that fail if a core idea is implemented wrong	68

STT: sunna `scratch/stt/`

FILE	LESSON	KYA KARTI HAI (FILE KI PEHLI LINE SE)	LINES
`audio.py`	1	audio: from a .wav file to the picture a speech model actually looks at	134
`text.py`	2	text: what the model writes, and how we score it	111
`data.py`	3	data: from a public speech corpus to arrays the training loop can read fast	157
`ctc.py`	4	CTC: how to train on (audio, text) pairs when nobody tells you WHEN each letter was said	190

FILE	LESSON	KYA KARTI HAI (FILE KI PEHLI LINE SE)	LINES
`model.py`	5	the model: an encoder that turns a spectrogram into per-frame character probabilities	273
`train.py`	6	training: the loop that turns random weights into a speech recognizer	196
`transcribe.py`	7	inference: a .wav file, or your own voice, to text	71
`evaluate.py`	8	evaluation: the number we report, and an honest yardstick next to it	94
`shrink.py`	9	quantization: the same idea as ../llm/shrink.py, measured on a speech model	65
`xray.py`	10	x-ray: watch CTC work on a real clip, frame by frame	80
`check.py`		Checks that fail if a core idea is implemented wrong	112

TTS: bolna `scratch/tts/`

FILE	LESSON	KYA KARTI HAI (FILE KI PEHLI LINE SE)	LINES
`audio.py`	1	audio, the other way round: from a spectrogram back to sound	146
`text.py`	2	the text front-end: what the model reads before it speaks	90
`data.py`	3	data: where a small TTS model gets clean speech to learn from	281
`align.py`	4	alignment: how long is each character spoken? Nobody told us that either	76
`model.py`	5	the acoustic model: characters in, a mel spectrogram out, all frames at once	258
`train.py`	6	training the acoustic model	204
`vocoder.py`	7	the vocoder: a small network that turns a mel spectrogram into a waveform	299
`speak.py`	8	inference: a sentence in, a .wav file out	82
`evaluate.py`	9	evaluation: can a machine understand what our model says?	93
`check.py`		Checks that fail if a core idea is implemented wrong	182

Orchestrator: baat-cheet `scratch/orchestrator/`

FILE	LESSON	KYA KARTI HAI (FILE KI PEHLI LINE SE)	LINES
`vad.py`	1	VAD: is anyone speaking right now?	116
`turn.py`	2	turn detection: has the person finished, or are they just thinking?	113
`models.py`	3	the three models behind three plain functions	188
`pipeline.py`	4	the pipeline: the state machine that turns three models into a conversation	181
`agent.py`	5	run it: talk to the three models you built	119
`check.py`		Checks for the orchestration logic	142

9 Interview: pitch aur 60 sawaal, beginner se advanced

har topic teen star mein · pehle simple jawab, phir wo jo bolna hai · source: `docs/questions.md`

30 SECOND MEIN

"I built a voice agent's four parts from scratch in PyTorch. A decoder-only LLM with my own BPE tokenizer. A Conformer-style speech recognizer trained with CTC, where I implemented the loss, beam search and forced alignment. A non-autoregressive TTS with monotonic alignment search and an inverse-STFT vocoder, distilled from a larger teacher. And an orchestrator with VAD, an LLM-based turn detector, streaming and barge-in. It covers Hindi, Marathi, English and Hinglish, the models total 24 M parameters, and every module has invariant tests."

JAWAB DENE KA TAREEKA

1. Pehle ek line mein seedha jawab. **2.** Phir "kyun". **3.** Phir apne project ki ek cheez ya ek number ("maine ye banaya aur ye naapa"). Teesri line hi tumhe ratta maarne walon se alag karti hai. Jo nahi pata, wahan bolo: "ye maine nahi banaya, par ye aise kaam karta hai."

LLM Beginner

1. LLM kya hai?

SIMPLE Ek machine jo "agla shabd batao" khel bahut accha khelti hai.

BOLO A neural network trained to predict the next token. Text generation is predicting, sampling and repeating.

2. Token kya hota hai?

SIMPLE Text ka tukda. Aam shabd poora, mushkil shabd 2-3 tukdon mein.

BOLO A subword unit. It keeps the vocabulary small while still covering any word. I built a BPE tokenizer for mine.

3. Model seekhta kaise hai?

SIMPLE Jawab deta hai, galti naapta hai, har knob ko thoda sahi taraf ghumata hai. Baar-baar.

BOLO Forward pass, compute the loss, backpropagate to get a gradient for every weight, and take a small step against it.

4. Embedding kya hai?

SIMPLE Har token ka "pata": numbers ki ek list. Milte-julte shabdon ke pate paas hote hain.

BOLO A learned vector per token. Similar tokens end up with similar vectors.

5. Temperature kya karta hai?

SIMPLE Kam = safe aur ek jaisa, zyada = creative par galat bhi.

BOLO It divides the logits before softmax. Low sharpens the distribution, high flattens it.

LLM Intermediate

6. Attention kaise kaam karta hai?

SIMPLE Har shabd baaki shabdon ko dekhta hai aur kaam wale se jaankari le leta hai.

BOLO Each token makes a query, key and value. The query-key dot product, after softmax, weights a sum of the values.

7. Causal mask kyun?

SIMPLE Taaki model agla shabd dekh ke nakal na kare.

BOLO It stops a position attending to the future, so I can train on every position at once without leaking the answer.

8. Position ka pata kaise chalta hai?

SIMPLE Har shabd ke vector ko uski jagah ke hisaab se ghuma dete hain.

BOLO I use RoPE: queries and keys are rotated by an angle that depends on position, so attention sees relative position.

9. Overfitting kaise pakdoge?

SIMPLE Purane paper pe pass, naye pe fail.

BOLO Training loss keeps falling while validation loss rises. I keep the checkpoint with the best validation loss.

10. KV cache kya hai?

SIMPLE Purane shabdon ka hisaab yaad rakhna, taaki har baar shuru se na padhna pade.

BOLO Caching each layer's keys and values so every new token costs one position, not the whole sequence. I test that cached and uncached outputs match.

LLM Advanced

11. Pre-norm aur RMSNorm kyun?

SIMPLE Deep network mein numbers ka size sambhalne ke liye, saste tareeke se.

BOLO Pre-norm keeps the residual path clean so gradients flow; RMSNorm is LayerNorm without mean subtraction, cheaper and equally stable.

12. LoRA kaise kaam karta hai?

SIMPLE Poori kitaab dobara nahi likhte, haashiye mein chhote notes chipkate hain.

BOLO Freeze the weights and learn a low-rank update B·A beside them. B starts at zero so step zero is the original model. I trained 0.9% of the parameters.

13. Quantization se kya khota hai?

SIMPLE Numbers ko chhota likhne se thodi baareeki jaati hai.

BOLO int8 per-channel cost me nothing in validation loss; at 4 bits per-channel mattered, and 2 bits broke the model. I measured each on the same batches.

14. Model galat baat kyun bolta hai?

SIMPLE Use sach nahi pata, sirf ye ki kaunsa shabd accha lagta hai agla.

BOLO It is trained for likely text, not true text, and has no way to verify. Grounding with retrieval is the usual fix.

15. Pehla loss kitna hona chahiye, aur kyun dekhte ho?

SIMPLE ln(vocabulary). Hamara 8.37 aaya, hona tha 8.32.

BOLO A random model spreads probability evenly, so the first loss should be about ln(vocab size). If it is not, the setup has a bug.

STT Beginner

16. Awaaz computer mein kaise hoti hai?

SIMPLE Hawa ke dabaav ki naap, ek second mein 16,000 baar. Numbers ki lambi list.

BOLO A waveform: amplitude samples at a sampling rate. I use 16 kHz, which captures speech up to 8 kHz.

17. Spectrogram kya hai?

SIMPLE Awaaz ki tasveer: kab kaunsi frequency kitni zor se baji.

BOLO The magnitude of a short-time Fourier transform over time. It makes phonetic content visible, which the raw waveform does not.

18. Mel scale kyun?

SIMPLE Kaan neeche ki frequencies mein baareek fark sunta hai, upar nahi.

BOLO It spaces frequency bands like human hearing: fine at the bottom, coarse at the top. 80 mel bands is the standard input.

19. WER kya hai?

SIMPLE 100 shabdon mein kitne galat.

BOLO Word error rate: the edit distance between hypothesis and reference in words, divided by the reference length. It can exceed 100%.

20. STT ko train karne ke liye kya chahiye?

SIMPLE Awaaz aur uska likha hua roop, saath-saath.

BOLO Paired audio and transcripts. It is supervised learning, which is why speech data is expensive.

STT Intermediate

21. CTC kya hal karta hai?

SIMPLE Kaunsa frame kis akshar ka hai, ye bataye bina training.

BOLO The alignment problem. The model emits a symbol per frame, and the loss sums the probability of all paths that collapse to the transcript.

22. Blank symbol kyun?

SIMPLE "Abhi kuch naya nahi" kehne ke liye, aur do "l" ko alag rakhne ke liye.

BOLO It lets the model emit nothing on a frame, and it separates genuine repeated characters, which would otherwise be merged.

23. Greedy aur beam search?

SIMPLE Greedy sabse accha raasta leta hai; beam sabse accha text.

BOLO Greedy takes the best symbol per frame. Prefix beam search keeps the best texts and adds up the paths that collapse to each.

24. Subsampling kyun karte ho?

SIMPLE 100 frames har second zyada hain; 25 kaafi hain aur attention sasta ho jaata hai.

BOLO Two stride-2 convolutions reduce 100 frames per second to 25. Attention cost falls 16 times and each frame covers a useful patch.

25. SpecAugment kya hai?

SIMPLE Training mein tasveer ki kuch pattiyan chhupa dena.

BOLO Masking random time and frequency stripes of the spectrogram. It is the main regulariser when speech data is small.

STT Advanced

26. CTC ki kamzori kya hai?

SIMPLE Har frame akela faisla leta hai; use yaad nahi ki pehle kya likha.

BOLO Conditional independence between frames: no internal language model. Transducers and attention decoders remove it at the cost of speed and simplicity.

27. CTC, RNN-T aur Whisper mein fark?

SIMPLE CTC sabse saral, RNN-T streaming ke liye, Whisper decoder wala.

BOLO CTC is encoder-only and non-autoregressive. RNN-T adds a prediction network over past outputs. Whisper is an attention encoder-decoder, most accurate but not streaming.

28. Padding se kya bug aa sakta hai?

SIMPLE Batch ki khaali jagah agar model dekh le to wahi clip alag jawab dega.

BOLO If convolutions or attention read padded frames, a clip's output depends on its batch. I mask in three places and test that alone and batched outputs match.

29. Training plateau pe atki, kya kiya?

SIMPLE Model sirf blank bol raha tha. Warmup lamba kiya.

BOLO It was the CTC blank plateau. A controlled experiment showed the warmup was too short; with 3000 steps it left the plateau.

30. Forced alignment kya hai?

SIMPLE Diye hue text ke liye pata karna ki kaunsa shabd kab bola gaya.

BOLO The Viterbi version of the CTC recursion: the single best path for a known transcript. It gives word timestamps without timing labels.

TTS Beginner

31. TTS ke do hisse kaunse hain?

SIMPLE Pehle text se tasveer (mel), phir tasveer se awaaz.

BOLO An acoustic model that predicts a mel spectrogram, and a vocoder that turns it into a waveform.

32. TTS, STT se mushkil kyun?

SIMPLE STT jaankari phenkta hai; TTS ko wo wapas banani padti hai.

BOLO It is one-to-many: one sentence can be spoken many ways, and the model must invent pitch, timing and phase.

33. Vocoder kya hai?

SIMPLE Mel ki tasveer se asli awaaz banane wala.

BOLO A model that generates the waveform from a mel spectrogram, which means reconstructing the phase the spectrogram dropped.

34. Duration kya hai?

SIMPLE Ek akshar kitni der bola jaata hai.

BOLO The number of mel frames each character or phoneme lasts. Predicting it is what lets a model generate all frames in parallel.

35. TTS mein punctuation kyun rakhte ho?

SIMPLE Comma rukna hai, sawaaliya nishaan uthta sur.

BOLO Punctuation carries prosody: pauses and pitch movement. Recognition drops it; synthesis needs it.

TTS Intermediate

36. Autoregressive aur non-autoregressive TTS?

SIMPLE Ek-ek frame banana, ya sab ek saath.

BOLO Tacotron generates frame by frame with attention and can skip or repeat words. FastSpeech-style models fix durations first and generate in parallel.

37. MAS kya hai?

SIMPLE Training ke dauraan khud pata karna ki har akshar ko kitne frames mile.

BOLO Monotonic alignment search: dynamic programming for the best monotonic text-to-frame alignment, used to get durations without an external aligner.

38. Length regulator?

SIMPLE Har akshar ka vector utni baar dohrana jitni uski duration.

BOLO It expands the text-rate sequence to frame rate by repeating each encoder output by its duration.

39. Griffin-Lim kya hai, aur kyun kaafi nahi?

SIMPLE Phase ka andaza lagane ka purana tareeka; jhanjhanata hai.

BOLO An iterative phase estimate from magnitudes only. It is consistent but not natural, which is why neural vocoders exist.

40. Characters ya phonemes?

SIMPLE Hum seedhe akshar dete hain; Devanagari lagbhag jaisa likha waisa bola jaata hai.

BOLO I use raw characters, like AI4Bharat's Indic TTS. The cost in Hindi is schwa deletion, which the model must learn from examples.

TTS Advanced

41. L1 loss se awaaz dhundhli kyun hoti hai?

SIMPLE Wo saare sambhav jawabon ka ausat banata hai.

BOLO L1 rewards the average of all plausible spectrograms, which is over-smoothed. Flow matching, diffusion or a GAN loss fix that.

42. HiFi-GAN aur Vocos mein fark?

SIMPLE Ek mel ko phula ke waveform banata hai, doosra frame rate pe rehke phase predict karta hai.

BOLO HiFi-GAN upsamples with transposed convolutions, so most compute is at audio rate. Vocos predicts magnitude and phase per frame and uses one inverse STFT.

43. Multi-resolution STFT loss kyun?

SIMPLE Chhoti window timing dekhti hai, lambi pitch.

BOLO Comparing spectrograms at several window sizes covers the time-frequency trade-off, and avoids comparing waveforms whose phases differ but sound the same.

44. Distillation kya hai, aur tumne kahan kiya?

SIMPLE Bada model bolta hai, chhota uski nakal seekhta hai.

BOLO Training a student on a teacher's outputs. My TTS training audio was generated by a 99 million parameter teacher; the student cannot exceed it.

45. TTS ko naapte kaise ho?

SIMPLE Log sunke number dete hain; ya machine se likhwa ke dekhte hain.

BOLO MOS from listeners for naturalness. For intelligibility I run the audio through a strong recognizer and compare its CER with the teacher's on the same sentences.

Orchestration Beginner

46. Voice agent ka pipeline kya hai?

SIMPLE Suno, likho, socho, bolo.

BOLO VAD, speech-to-text, turn detection, an LLM, sentence chunking and text-to-speech: a cascaded pipeline.

47. VAD kya hai?

SIMPLE Mic pe koi bol raha hai ya nahi.

BOLO Voice activity detection: it marks where an utterance starts and ends, so the recognizer only gets speech.

48. Latency kya hai aur kyun zaroori?

SIMPLE User ke rukne aur agent ke bolne ke beech ka samay.

BOLO Time to first audio. Above roughly a second a conversation stops feeling natural.

49. Barge-in kya hai?

SIMPLE User beech mein bole to agent turant chup.

BOLO Interrupting the agent while it speaks: playback is cut and the rest of the reply is cancelled.

50. Streaming kyun?

SIMPLE Poore jawab ka intezaar karoge to der hogi.

BOLO Each stage starts on partial output of the one before, so the first sentence is spoken while the rest is still being generated.

Orchestration Intermediate

51. End-of-turn detection mushkil kyun?

SIMPLE Beech ka rukna aur baat ka ant dono chuppi hain.

BOLO Silence alone cannot tell a pause from the end. I combine silence duration with a semantic signal from the LLM.

52. Hysteresis kya hai VAD mein?

SIMPLE Shuru hone aur rukne ke alag niyam, taaki faisla baar-baar na palte.

BOLO Different thresholds for starting and stopping: 100 ms of speech to start, 500 ms of silence to end, plus a pre-roll buffer.

53. Latency kahan-kahan jaati hai?

SIMPLE Chuppi ka intezaar, STT, LLM ka pehla tukda, TTS ka pehla vaakya.

BOLO Endpointing delay, STT time, LLM time to first token and TTS time for the first sentence. I log each per turn.

54. Model calls thread mein kyun?

SIMPLE Warna sochte waqt agent behra ho jaayega.

BOLO They block. Running them in worker threads keeps the event loop reading the microphone, which is what makes barge-in possible.

55. Sentence chunking kyun?

SIMPLE TTS ko poora vaakya chahiye, par poore jawab ka intezaar nahi.

BOLO Prosody needs whole sentences, so I regroup the token stream into sentences and synthesize each as soon as it closes.

Orchestration Advanced

56. Cancellation sahi kaise karte ho?

SIMPLE Har jawab ka number; purana number wala kaam khud ruk jaata hai.

BOLO A generation counter. Threads cannot be killed, so every stage checks that its generation is still current before acting.

57. False cutoff aur latency ka trade-off?

SIMPLE Jaldi jawab doge to tokne ka khatra badhta hai.

BOLO A shorter endpointing delay cuts latency but raises false cutoffs. I use a short delay when the text looks finished and a long one when it does not.

58. Echo ka kya karte ho?

SIMPLE Speaker ki awaaz mic mein jaati hai; agent khud ko tok dega.

BOLO It needs acoustic echo cancellation. Browsers and WebRTC provide it; my local script does not, so it needs headphones.

59. WebRTC kyun, WebSocket kyun nahi?

SIMPLE WebRTC der se aaye packet ko chhod deta hai; WebSocket atak jaata hai.

BOLO WebRTC runs on UDP with a jitter buffer, loss concealment and echo cancellation. WebSocket is TCP, so one lost packet stalls everything behind it.

60. Orchestrator ko test kaise kiya?

SIMPLE Asli models ki jagah nakli functions, tay delay ke saath.

BOLO Fake models with known delays, and scenario tests: a full turn, a barge-in, a pause that is not the end, and a model that raises.

Aakhri sawaal: ye production mein chalega?

SIMPLE Nahi. Ye seekhne ke models hain. Sach bolna hi sahi jawab hai.

BOLO No. They are small and trained on tens of hours. The point was to understand every part; in production I would fine-tune pretrained models.

10 Sachchi baatein aur agla kadam

jo nahi bana, aur kab banana

KYA NAHI HAI	KYUN	AGLA KADAM
Bade models jaisi accuracy	23.5 ghante data aur 4.6 M parameters. Whisper ne 680,000 ghante dekhe.	Zyada data, ya bade model ke pseudo-labels
Streaming STT	Model poora clip ek saath dekhta hai	Sirf peeche dekhne wala encoder
TTS mein pitch control	Sirf duration predict hoti hai	Pitch predictor (FastPitch)
GAN wala vocoder	Sirf spectrum ka loss hai, to halki "buzz" reh sakti hai	Discriminator jodna
Numbers bolna ("2024")	Text front-end digits nahi kholta	Number-to-words ka kadam
Echo cancellation	Networking / signal ka kaam hai	Browser ye muft deta hai
Browser mein chalna	Models ko ONNX mein nikaalna baaki hai	ONNX export, phir static page pe free hosting
Baat-cheet ki yaad	Scratch LLM ek baari ka hai	Pichhli baariyan prompt mein jodna

LICENSE KI EK BAAT

TTS ne Supertonic 3 ki awaaz se seekha hai. Uske Open RAIL-M license ke hisaab se hamara model uska "derivative" hai: uski use-restrictions (jaise kisi ki nakal karke dhokha na dena, aur batana ki awaaz machine ki hai) hamare weights ke saath chalti hain.

SEEKHNE KA KRAM, AGAR 7 DIN HON

Din 1-2: LLM ki zero guide. **Din 3:** chapters 1-4, saath mein `stt/audio.py` aur `stt/ctc.py` chalao. **Din 4:** `stt/model.py` aur training ka log. **Din 5:** TTS, `align.py` ka demo haath se karo. **Din 6:** orchestrator, `check.py` ke scenarios padho. **Din 7:** 20 sawaal bina dekhe bolo.

Shabdkosh: 410 terms

har term ke saamne wo jagah jahan tumne use code mein banaya · LLM ke 169 terms uski apni guide mein hain

STT glossary

A. Awaaz ki neenv (lesson 1: audio.py)

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
1	**Waveform**	Hawa ke dabaav ka time ke saath record. Numbers ki ek lambi list, -1 se 1 ke beech.	`audio.py def read_wav`
2	**Sample**	Us list ka ek number: ek pal ka dabaav.	`audio.py np.frombuffer`
3	**Sampling rate**	Ek second mein kitne samples. Hamara 16,000 (16 kHz).	`audio.py SR = 16000`
4	**Nyquist limit**	Sampling rate ka aadha hi sabse oonchi frequency hai jo pakdi ja sakti hai. 16 kHz pe 8 kHz tak.	`audio.py Nyquist`
5	**WAV / RIFF**	Audio file ka sabse seedha format: chhota header, phir samples.	`audio.py b"RIFF"`
6	**Chunk (WAV)**	WAV file ke tukde: naam, size, bytes. `fmt ` mein format, `data` mein samples.	`audio.py name == b"fmt "`
7	**PCM 16-bit**	Har sample ek 16-bit integer (-32768 se 32767). 32768 se bhaag do to -1..1.	`audio.py "<i2"`
8	**Float audio**	Har sample seedha 32-bit float. FLEURS isi mein hai.	`audio.py "<f4"`
9	**Mono / stereo**	Ek channel ya do. Speech model ko ek chahiye, to channels ka average.	`audio.py .mean(axis=1)`
10	**Frequency**	Ek second mein kitni baar dabaav upar-neeche hota hai (Hz). Patli awaaz = zyada.	`audio.py peak_hz`
11	**Frame / window**	Waveform ka chhota tukda (25 ms) jisme awaaz "sthir" maani jaati hai.	`audio.py N_FFT = 400`
12	**Hop length**	Agla frame kitne samples baad shuru. 160 = 10 ms, yaani 100 frames/second.	`audio.py HOP = 160`
13	**Overlap**	Frames ek doosre pe chadhe hote hain (25 ms lamba, 10 ms pe naya). Kuch chhoot-ta nahi.	`audio.py wave.unfold`
14	**Fourier transform**	Signal ko uski frequencies mein todna: "ismein kaunsi frequency kitni hai".	`audio.py torch.fft.rfft`
15	**FFT**	Fourier transform nikalne ka tez algorithm.	`audio.py torch.fft.rfft`
16	**STFT**	Short-time Fourier transform: har chhote frame ka alag FFT. Time aur frequency dono milte hain.	`audio.py def power_spectrogram`
17	**FFT bin**	FFT ka ek output: ek frequency ki patti. 400 samples pe bins 40 Hz door hain.	`audio.py SR / N_FFT`

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
18	**Hann window**	Frame ke kinaaron ko dheere se zero karna, taaki kata hua kinara jhoothi frequency na banaye.	`audio.py torch.hann_window`
19	**Spectral leakage**	Wo jhoothi frequencies jo frame ko seedha kaatne se aati hain.	`audio.py spectral leakage`
20	**Magnitude / phase**	FFT ka har number complex hai: kitna zor (magnitude) aur lehar kahan se shuru (phase).	`audio.py spectrum.abs()`
21	**Power spectrogram**	Magnitude ka square, har frame aur har bin ke liye. Phase phenk diya.	`audio.py spectrum.abs().pow(2)`
22	**Mel scale**	Frequency ka wo scale jo kaan jaisa hai: neeche baareek, upar mota.	`audio.py def hz_to_mel`
23	**Mel filterbank**	80 triangles jo 201 bins ko 80 bands mein samet dete hain.	`audio.py def mel_filterbank`
24	**Mel bands**	Filterbank ke baad ke 80 numbers.	`audio.py N_MELS = 80`
25	**Log compression**	Energy ka log lena. Kaan anupaat sunta hai, antar nahi.	`audio.py torch.log(mel.clamp`
26	**Log-mel spectrogram**	Upar ka sab milake: (frames, 80) ki tasveer. Model ka input.	`audio.py def log_mel`
27	**Reflect padding**	Shuru aur aakhir mein signal ko aaine ki tarah badhana, taaki frame t theek t pe centred ho.	`audio.py mode="reflect"`
28	**Chirp**	Aisi tone jiski frequency badhti jaati hai. Spectrogram check karne ka tareeka.	`audio.py sweep = torch.sin`
29	**MFCC**	Log-mel pe ek aur transform (DCT). Purane HMM systems ka feature; neural models log-mel hi lete hain.	theory
30	**Pre-emphasis**	Oonchi frequencies ko thoda badhana. Purana trick, ab zaroori nahi.	theory
31	**Resampling**	Sampling rate badalna (jaise 48 kHz se 16 kHz). Pehle low-pass filter, warna aliasing.	theory
32	**Aliasing**	Nyquist se oonchi frequency ka galat, neechi frequency ban ke dikhna.	theory

B. Text aur scoring (lesson 2: text.py)

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
33	**Transcript**	Clip mein jo bola gaya uska likha hua roop. Training ka label.	`data.py "text": text`
34	**Text normalization**	Transcript se wo sab hatana jo suna nahi ja sakta: punctuation, capital letters.	`text.py def normalize`
35	**Unicode NFC**	Ek hi dikhne wale akshar ko ek hi tarah store karna.	`text.py unicodedata.normalize("NFC"`

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
36	**Code point**	Unicode ka ek number. "कि" do code points hai: क aur ि.	`text.py code points of`
37	**Matra**	Devanagari mein swar ka chinh (ा ि ी ...). Alag code point hai.	`text.py unicodedata.category`
38	**Character vocabulary**	Model jo symbols likh sakta hai unki list. Hamari lagbhag 110.	`text.py class Vocab`
39	**Grapheme**	Likhne ki ikai (akshar). Hum graphemes predict karte hain, phonemes nahi.	`text.py def encode`
40	**Phoneme**	Bolne ki ikai (dhwani). "phone" mein 5 graphemes, 3 phonemes.	theory
41	**Blank symbol**	CTC ka extra symbol: "is frame mein kuch naya nahi". Id 0.	`text.py BLANK = 0`
42	**Rare character cutoff**	Jo character 5 se kam baar aaya use vocabulary se hata dena: seekha nahi ja sakta.	`text.py min_count`
43	**Edit distance**	Ek text ko doosre mein badalne ke liye kam se kam kitne insert / delete / substitute.	`text.py def edit_distance`
44	**WER**	Word error rate: shabdon pe edit distance / reference ke shabd.	`text.py def error_rate`
45	**CER**	Character error rate: wahi, characters pe. Lambe shabdon wali bhashaon ke liye zyada nyaay-sangat.	`text.py unit: str = "word"`
46	**Substitution / deletion / insertion**	WER ki teen galtiyan: galat shabd, chhoota shabd, faltu shabd.	`text.py diagonal + (r != h)`
47	**WER > 100%**	Ho sakta hai: model itne faltu shabd jode ki galtiyan reference se zyada ho jaayen.	`text.py It can exceed 1.0`
48	**Subword (BPE) for ASR**	Bade models characters ki jagah subwords likhte hain. Kam data pe characters behtar.	theory

C. Data (lesson 3: data.py)

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
49	**Supervised learning**	Har input ke saath insaan ka diya sahi jawab. Speech ke liye: audio + transcript.	`data.py SUPERVISED`
50	**Corpus**	Data ka sangrah. Hamara FLEURS.	`data.py google/fleurs`
51	**Read speech**	Likha hua padhke bola gaya. Saaf hota hai; asli baat-cheet (spontaneous) mushkil.	`data.py reading Wikipedia sentences`
52	**Train / dev / test split**	Seekhne ke liye / training ke dauraan dekhne ke liye / aakhir mein ek baar naapne ke liye.	`data.py ("dev", "test", "train")`
53	**Sentence-disjoint split**	Test ka koi vaakya train mein nahi. Warna model ratta maar ke pass ho jaata.	`data.py "sentence": row[0]`

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
54	**Feature extraction**	Audio se log-mel banana. Har epoch same hai, to ek baar karke disk pe rakh do.	`data.py def build_split`
55	**Streaming a tar**	Archive ko kholke ek-ek file padhna, disk pe failaye bina.	`data.py tarfile.open`
56	**float16 storage**	Features ko aadhe size mein rakhna. Model ko fark nahi padta.	`data.py np.float16`
57	**Memmap**	Badi file ko RAM mein laaye bina array ki tarah padhna.	`data.py np.memmap`
58	**Index**	Har clip ke liye: frames kahan se shuru, kitne, text, bhasha.	`data.py def open_split`
59	**Max duration filter**	Bahut lambe training clips hatana: ek lamba clip poore batch ki memory tay karta hai.	`data.py MAX_TRAIN_SEC`
60	**Feature mean / std**	Har mel band ka ausat aur failaav, sirf train pe naapa.	`data.py mean = total / len(mel)`
61	**Data leakage**	Test ki jaankari ka training mein ghus jaana (jaise stats test pe naapna). Score jhootha ho jaata hai.	`data.py leak information`
62	**Hours of speech**	Speech data ghanton mein naapa jaata hai. Hamara ~27 h; Whisper ka 680,000 h.	`data.py seconds / 3600`
63	**Pseudo-labeling**	Bade model se bina-label audio ka transcript banwa ke us pe chhota model train karna.	theory
64	**Speed perturbation**	Audio ko 0.9x / 1.1x speed pe chalake data badhana.	theory

D. CTC (lesson 4: ctc.py)

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
65	**Alignment problem**	Kaunsa frame kis akshar ka hai, ye kisi ne nahi bataya.	`ctc.py Which frame belongs to which character`
66	**CTC**	Connectionist Temporal Classification: bina alignment ke audio-text pe train karne ka tareeka.	`ctc.py def ctc_loss`
67	**Path**	Har frame ke liye ek symbol: model ka frame-dar-frame output.	`ctc.py frame-by-frame output`
68	**Collapse rules**	Pehle repeats milao, phir blanks hatao. Path se text.	`ctc.py torch.unique_consecutive`
69	**Blank ka doosra kaam**	Asli double letters ko alag rakhna: "l-l" → "ll", "ll" → "l".	`ctc.py can_skip`
70	**Extended label sequence**	Target ke aage, beech aur peeche blank: "hi" → -h-i-. 2L+1 states.	`ctc.py def _extend`
71	**Marginalization**	Sahi text dene wale SAARE paths ki probability jodna.	`ctc.py torch.logsumexp`
72	**Forward algorithm**	Wo jod dynamic programming se, T × (2L+1) steps mein.	`ctc.py alpha`

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
73	**Alpha**	Frame t tak, state s mein pahunchne wale saare adhoore paths ki log-probability.	`ctc.py stay = alpha`
74	**Stay / step / skip**	State mein aane ke teen raaste: wahi rukna, ek aage, ya blank chhod ke do aage.	`ctc.py step = torch.cat`
75	**Emission**	Kisi frame pe kisi symbol ki log-probability.	`ctc.py emit = log_probs.gather`
76	**Log space**	Probabilities ki jagah unke log. Chhoti probabilities ka guna zero ban jaata hai (underflow).	`ctc.py def _log_add`
77	**log(0) ka jugaad**	Asli -inf se gradient NaN ho jaata hai, isliye bahut bada negative number.	`ctc.py NEG = -1e30`
78	**Input length masking**	Batch mein chhota clip apne asli frames ke baad update hona band kar deta hai.	`ctc.py in_lens`
79	**Conditional independence**	CTC ki maanyata: har frame ka output baaki outputs se azaad hai (audio diya ho to).	`ctc.py INDEPENDENTLY`
80	**Greedy / best-path decoding**	Har frame ka sabse sambhav symbol, phir collapse.	`ctc.py def greedy`
81	**Prefix beam search**	Sabse sambhav TEXTS rakhna, paths nahi. Ek text ke kai paths ki probability judti hai.	`ctc.py def beam_search`
82	**Beam width**	Har frame ke baad kitne candidate texts zinda rakhne hain.	`ctc.py beam: int = 8`
83	**p_blank / p_non_blank**	Har prefix ke do numbers: blank pe khatm hone wale paths, aur akshar pe khatm hone wale.	`ctc.py p_b, p_nb`
84	**Pruning**	Bahut kam probability wale symbols ko dekhna hi nahi.	`ctc.py frame.topk`
85	**Forced alignment**	Diye hue text ke liye sabse sambhav path: kaunsa akshar kab bola gaya.	`ctc.py def align`
86	**Viterbi**	Forward algorithm mein "sum" ki jagah "max", saath mein peeche jaane ka record.	`ctc.py moves[s] = max`
87	**Spiky output**	Trained CTC zyada-tar blank bolta hai aur har akshar ek-do frame mein.	`xray.py spiky`
88	**LM fusion**	Beam search mein language model ka score jodna, taaki spelling aur grammar sudhre.	theory
89	**RNN-T / transducer**	CTC ka bada bhai: output pichhle outputs pe bhi nirbhar. Streaming ASR ka standard.	theory
90	**Attention encoder-decoder**	Whisper ka tareeka: decoder ek-ek token likhta hai, encoder pe attention karke.	theory

E. Model (lesson 5: model.py)

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
91	**Acoustic model / encoder**	Spectrogram ko har frame ke liye character probabilities mein badalne wala network.	`model.py class Encoder`
92	**Encoder-only**	Koi decoder nahi, koi loop nahi: ek pass mein saare frames.	`model.py ENCODER-ONLY`
93	**Feature normalization**	Har mel band ko mean 0, std 1 pe laana, model ke andar.	`model.py (mel - self.mean) / self.std`
94	**Buffer in checkpoint**	Mean / std weights ke saath save hote hain, to model hamesha waise hi normalize karega jaise train hua.	`model.py self.register_buffer("mean"`
95	**Subsampling**	100 frames/s ko 25 pe laana. Attention 16 guna sasta.	`model.py class Subsample`
96	**Strided convolution**	Aisa convolution jo har doosri position chhod deta hai: output aadha.	`model.py stride=2`
97	**Output length formula**	Convolution ke baad kitne frames bachenge: (n - 1) // 2 + 1. CTC ko ye sahi chahiye.	`model.py def out_len`
98	**Frame rate**	Model ke output ki raftaar: 25 frames/s, ek frame = 40 ms.	`xray.py FRAME_SEC = 0.04`
99	**Padding**	Batch ke chhote clips ko lambe ke barabar karne ke liye joda gaya khaali hissa.	`train.py pad_sequence`
100	**Padding mask**	Bool array: kaunsa frame asli hai, kaunsa padding.	`model.py def frame_mask`
101	**Attention padding mask**	Koi frame padding pe attention na de.	`model.py ~real[:, None, None, :]`
102	**Padding invariance**	Clip akela ho ya batch mein, output same. Na ho to batch size badalte hi accuracy badlegi.	`model.py SAME`
103	**Bidirectional attention**	Har frame poora clip dekh sakta hai, aage ka bhi. LLM mein ye mana tha.	`model.py class SelfAttention`
104	**Conformer**	Transformer + convolution ka block. Speech ka standard encoder (2020).	`model.py class Block`
105	**Squeezeformer layout**	Conformer ka saral roop: attention → conv → feed-forward, seedhe pre-norm residuals.	`model.py x = x + self.conv`
106	**Convolution module**	Aas-paas ke 15 frames ko milana: local pattern pakadta hai.	`model.py class ConvModule`
107	**Depthwise convolution**	Har channel ka apna alag chhota filter. 15 × d weights, 15 × d × d nahi.	`model.py groups=cfg.d_model`
108	**Pointwise layer**	Har frame pe akele lagne wali linear layer: channels ko milati hai.	`model.py self.expand`
109	**GLU gate**	Aadhe channels sigmoid se guzar ke baaki aadhon ko rokte / jaane dete hain.	`model.py torch.sigmoid(gate)`

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
110	**Kernel size**	Convolution ek baar mein kitne frames dekhta hai. 15 × 40 ms = 0.6 s.	`model.py conv_kernel: int = 15`
111	**Receptive field**	Ek output kitne input ko dekh paata hai. Conv se local, attention se poora clip.	`model.py global context`
112	**RoPE on frames**	LLM wala hi rotary position, yahan tokens ki jagah frames pe.	`model.py def apply_rope`
113	**SpecAugment**	Training mein spectrogram ki random pattiyan chhupa dena. Speech ka sabse zaroori augmentation.	`model.py def spec_augment`
114	**Frequency masking**	Kuch mel bands ko poore clip mein zero karna.	`model.py freq_masks`
115	**Time masking**	Kuch pal ke saare bands zero karna.	`model.py time_masks`
116	**CTC head**	Aakhri linear layer: d numbers se vocabulary ke scores.	`model.py self.head = nn.Linear`
117	**Log-softmax**	Scores ko log-probabilities mein. CTC ko yahi chahiye.	`model.py F.log_softmax`
118	**Safe checkpoint loading**	Sirf tensors load karna, pickle ka code nahi chalana.	`model.py weights_only=True`
119	**Streaming encoder**	Aisa encoder jo sirf peeche (aur thoda aage) dekhta hai, taaki bolte-bolte likh sake. Hamara nahi hai.	theory
120	**Lookahead**	Streaming model kitna aage ka audio dekhta hai. Moonshine v2 mein 80 ms.	theory
121	**Self-supervised pretraining**	wav2vec 2.0 / HuBERT: bina transcript ke audio se seekhna, phir thode labels pe fine-tune.	theory
122	**Vector quantization**	Continuous vector ko codebook ke sabse kareebi entry se badalna. Codecs aur wav2vec mein.	theory

F. Training (lesson 6: train.py)

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
123	**Overfit sanity test**	Pehle 16 clips ratwa ke dekho. Na ratein to code mein bug hai, data mein nahi.	`train.py --overfit`
124	**Length bucketing**	Ek jaisi lambai ke clips ek batch mein, taaki padding kam ho.	`train.py def make_batches`
125	**Dynamic batch size**	Batch clips ki ginti se nahi, kul frames se tay: chhote clips ke bade batch.	`train.py max_frames`
126	**Collate**	Alag lambai ke clips ko ek padded tensor mein jodna, saath mein asli lambaiyan.	`train.py def collate`
127	**CTC loss per character**	Har clip ka loss uske characters se bhaag, phir ausat. Lambe clips haavi nahi hote.	`train.py F.ctc_loss`

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
128	**Infeasible target**	Jab text mein output frames se zyada characters hon: koi valid path nahi, loss infinity.	`train.py zero_infinity=True`
129	**Blank plateau**	Shuru mein model "hamesha blank" seekh leta hai aur loss atak jaata hai, phir achanak likhne lagta hai.	`train.py PLATEAU`
130	**Dev CER se checkpoint chunna**	Jo checkpoint an-dekhe clips pe sabse accha, wahi rakho.	`train.py if cer < best`
131	**Macro average**	Har bhasha ka CER barabar wazan se jodna, taaki zyada data wali bhasha haavi na ho.	`train.py each counts equally`
132	**Resume**	Ruki hui training wahin se chalana. Optimizer ki state bhi chahiye.	`train.py opt.load_state_dict`
133	**Shuffling batches**	Har epoch batches ka order badalna.	`train.py random.shuffle(batches)`
134	**Augmentation sirf train mein**	SpecAugment training mein on, evaluation mein off.	`train.py augment=not args.overfit`
135	**Epoch**	Poore training data ka ek chakkar.	`train.py for epoch in range`

G. Inference, evaluation, size (lessons 7-10)

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
136	**Non-autoregressive inference**	Poora clip ek pass mein. LLM jaisa token-dar-token loop nahi.	`transcribe.py def transcribe`
137	**Real-time factor (RTF)**	Compute ke second / audio ke second. 1 se kam = bolne se tez.	`transcribe.py real-time factor`
138	**Microphone capture**	Mic se seedha 16 kHz mono float samples lena.	`transcribe.py sounddevice.rec`
139	**Test set discipline**	Test split ko training ke dauraan kabhi nahi dekhna.	`evaluate.py open_split("test")`
140	**Baseline**	Wo jaana-maana model jiske saamne apna number rakhte hain.	`evaluate.py from_omnilingual_asr_ctc`
141	**Fair comparison**	Same clips, same normalization, same metric, same threads.	`evaluate.py torch.set_num_threads(2)`
142	**Post-training quantization**	Train hue weights ko kam bits mein rakhna aur nuksaan naapna.	`shrink.py def quant_roundtrip`
143	**Per-channel scale**	Har output filter ka apna scale. Conv aur linear dono pe.	`shrink.py per_channel`
144	**Confidence**	Model apne chune hue symbol ko kitni probability deta hai.	`xray.py confidence`
145	**Word timestamps**	Har shabd kab se kab tak bola gaya, forced alignment se.	`xray.py align(log_probs`

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
146	**Endpointing**	Ye tay karna ki bolne wala ruk gaya. Orchestrator ka kaam (module 4).	theory
147	**VAD**	Voice activity detection: is pal koi bol raha hai ya nahi.	theory
148	**Streaming vs batch ASR**	Bolte-bolte likhna, ya poora turn sunke. Hamara batch hai.	theory
149	**Hallucination (ASR)**	Decoder wale models chup audio pe bhi vaakya likh dete hain. CTC mein ye bahut kam.	theory
150	**Diarization**	"Kisne kab bola". ASR se alag kaam.	theory
151	**Code-switching**	Ek hi vaakya mein do bhashayen (Hinglish). ASR ke liye mushkil: do scripts, ek awaaz.	theory
152	**Knowledge distillation**	Bade model (teacher) ke outputs se chhote model (student) ko sikhana.	theory

TTS glossary

A. Awaaz wapas banana (lesson 1: audio.py)

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
1	**Speech synthesis / TTS**	Text se awaaz banana. STT ka ulta, aur mushkil: jo jaankari phenki thi wo wapas laani hai.	`speak.py def speak`
2	**22.05 / 24 kHz**	TTS ka sampling rate. Samajhne ke liye 16 kHz kaafi, achha sunne ke liye zyada chahiye. Lesson 10: `TTS_SR=24000` (100 mel bands).	`audio.py TTS_SR`
3	**Lamba window**	46 ms ka frame: frequency ki baareeki zyada (pitch ke harmonics dikhte hain).	`audio.py N_FFT = 1024`
4	**Frame rate**	86 mel frames per second.	`audio.py HOP = 256`
5	**Resampling**	Sampling rate badalna (44.1 → 22.05 kHz).	`audio.py def resample`
6	**Sinc interpolation**	Har sample pe ek sinc rakh ke unka jod: samples ke beech ka sahi curve.	`audio.py torch.sinc`
7	**Anti-aliasing filter**	Rate ghatane se pehle oonchi frequencies kaatna.	`audio.py cutoff`
8	**Aliasing**	Na kaato to oonchi frequency jhoothi neechi tone ban jaati hai (15 kHz → 7 kHz).	`audio.py aliasing`
9	**Windowed sinc**	Asli sinc anant lamba hai; use window se kaat-te hain.	`audio.py width`
10	**WAV likhna**	Wahi header jo STT ne padha tha, ab haath se likha.	`audio.py def write_wav`
11	**Silence trimming**	Clip ke shuru / aakhir ki chuppi kaatna.	`audio.py def trim_silence`
12	**Magnitude spectrogram**	TTS mein power nahi, magnitude ka log lete hain (vocoders ka riwaaz).	`audio.py def log_mel`

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
13	**Log floor**	log(0) se bachne ke liye sabse chhoti value.	`audio.py clamp(min=1e-5)`
14	**Phase**	Har frequency ki lehar apne chakkar mein kahan hai. Spectrogram mein nahi hota.	`audio.py PHASE`
15	**Phase reconstruction**	Magnitude se phase ka andaza lagana: TTS ke aakhri kadam ki asli mushkil.	`audio.py def griffin_lim`
16	**Griffin-Lim**	Random phase se shuru, baar-baar: waveform banao, dobara analyse karo, uska phase rakho.	`audio.py again / again.abs()`
17	**Iterations**	Jitne zyada chakkar, utna self-consistent phase.	`audio.py rounds`
18	**Inverse STFT**	Complex spectrogram se waveform: overlap-add.	`audio.py torch.istft`
19	**Mel inversion**	80 bands se 513 bins wapas: pseudo-inverse se sabse accha andaza.	`audio.py torch.linalg.pinv`
20	**Metallic artifact**	Griffin-Lim ki pehchaan: galat phase ki jhanjhanahat.	`audio.py metallic`
21	**Harmonics**	Pitch ke guna (150, 300, 450 Hz ...). Awaaz ki "banawat".	`audio.py harmonics`
22	**Pitch / F0**	Vocal cords kitni baar per second khulte hain. Sur.	theory
23	**Formants**	Munh ki banawat se bani oonchi-energy wali pattiyan. Swar inse pehchana jaata hai.	theory
24	**Overlap-add**	Frames ko window lagake jodna, taaki jod pe awaaz na toote.	theory

B. Text front-end (lesson 2: text.py)

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
25	**Text front-end**	Text ko model ke layak banana: normalize, characters, bhasha.	`text.py def normalize`
26	**Punctuation as prosody**	Comma = chhota rukna, full stop = lamba aur girta sur. Isliye TTS use rakhta hai.	`text.py PUNCT`
27	**Punctuation unification**	। ; : — sab ko , ya . mein badalna.	`text.py _UNIFY`
28	**Character input**	Model seedha akshar padhta hai, phonemes nahi.	`text.py Characters, not phonemes`
29	**G2P**	Grapheme-to-phoneme: likhe ko bolne ki dhwaniyon mein badalna. Purane TTS ka pehla kadam.	`text.py G2P`
30	**Schwa deletion**	Hindi mein chhupa "a" kai jagah nahi bola jaata: कमल = "kamal".	`text.py schwa`
31	**Language id**	Wahi akshar alag bhasha mein alag bolte hain; model ko bhasha batani padti hai.	`text.py LANGS`
32	**Language guessing**	Script aur kuch pehchaan-shabdon se bhasha ka andaza.	`text.py def guess_lang`

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
33	**Hinglish**	Roman mein likhi Hindi, English shabdon ke saath. Apni alag id.	`text.py _HINGLISH`
34	**Hindi vs Marathi**	Ek script, alag ucharan: marker shabdon se alag karte hain.	`text.py _MARATHI`
35	**Padding id**	Batch bharne wala khaali symbol.	`text.py PAD = 0`
36	**Vocabulary**	Akshar jo model padh sakta hai; an-dekhe chhod diye jaate hain.	`text.py class Vocab`
37	**Text normalization (numbers)**	"2024" ko "do hazaar chaubees" banana. Hamare paas nahi hai.	`data.py digits`
38	**Homograph**	Ek spelling, do ucharan ("read"). G2P ki purani mushkil.	theory
39	**Phoneme set / IPA**	Dhwaniyon ki maanak list.	theory
40	**espeak-ng**	Open-source G2P jo Piper aur Kokoro use karte hain.	theory
41	**Code-switching in TTS**	Ek vaakya mein do bhashayen: ek awaaz mein dono bolna.	theory

C. Data (lesson 3: data.py)

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
42	**TTS data ki zaroorat**	Ek hi awaaz, saaf recording, ghanton tak. ASR data se ulta.	`data.py ONE consistent voice`
43	**Knowledge distillation**	Bada model (teacher) bolta hai, chhota (student) uski nakal seekhta hai.	`data.py Teacher (distillation)`
44	**Teacher model**	Supertonic 3 (99 M).	`data.py TEACHER`
45	**Synthetic data**	Model ka banaya training data. Jitna chaho, bilkul saaf.	`data.py def _speak`
46	**Teacher ke denoising steps**	Zyada steps = saaf target. Data ek hi baar banta hai, to yahan kanjoosi nahi.	`data.py config.num_steps`
47	**Student ki seema**	Student teacher ke paas pahunch sakta hai, usse aage nahi.	`data.py not beat it`
48	**Derivative model**	License ki bhasha: teacher ke output pe train hua model uska "derivative" hai.	`data.py derivative`
49	**Real recordings**	Marathi ke liye asli insaani awaaz (OpenSLR 64).	`data.py def marathi`
50	**Loudness normalization**	Har clip ko ek hi LOUDNESS (RMS -20 dBFS) pe laana, peak pe nahi: warna dheemi aur cheekhti clip ek jaisi zor se lagti hain.	`data.py -20 dBFS`
51	**PCM storage**	Samples ko seedha 16-bit integers mein rakhna.	`data.py .pcm`
52	**Parallel workers**	Paanch processes, har ek ke paas teacher ki apni copy. Ek worker mare to `ProcessPoolExecutor` zor se fail hota hai; `multiprocessing.Pool` hamesha intezaar karta reh jaata.	`data.py ProcessPoolExecutor(args.workers)`

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
53	**Held-out dev sentences**	Har bhasha ke 40 vaakya training se bahar.	`data.py DEV_PER_LANG`
54	**Sentence selection**	Na bahut chhote, na bahut lambe, bina digits ke.	`data.py def _usable`
55	**Text diversity**	Kahaniyon ke aasaan shabd + khabron ke mushkil shabd aur naam.	`data.py def texts_hi`
56	**Single vs multi-speaker**	Ek awaaz seekhna aasaan; kai ke liye speaker embedding chahiye.	theory
57	**Speaker embedding**	Awaaz ki pehchaan ka vector.	theory
58	**Voice cloning / zero-shot TTS**	Kuch second ke sample se nayi awaaz.	theory

D. Alignment aur duration (lesson 4: align.py)

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
59	**Duration**	Ek akshar kitne mel frames tak bola gaya.	`align.py durations`
60	**TTS ka alignment problem**	52 akshar se 430 frames: kaunsa akshar kitna lamba? Data mein nahi likha.	`align.py Durations are not in the data`
61	**Monotonic alignment**	Akshar kram mein, koi chhoota nahi, har frame ek hi akshar ka.	`align.py MONOTONIC`
62	**MAS**	Monotonic Alignment Search: sabse accha monotonic raasta, dynamic programming se.	`align.py def mas`
63	**Score table**	Har (akshar, frame) jodi ke liye: kitna milta hai.	`model.py score = -0.5`
64	**Stay / step**	Raasta ya usi akshar pe rukta hai ya agle pe jaata hai. CTC jaisa "skip" nahi.	`align.py step = np.concatenate`
65	**DP table**	best[l, t]: frame t tak, akshar l pe khatm hone wala sabse accha raasta.	`align.py best[:, :, t]`
66	**Backtracking**	Ant se peeche chal ke raasta padhna.	`align.py came_by_step`
67	**Durations from a path**	Raaste ne har akshar ko kitne frames diye, wahi uski duration.	`align.py += 1`
68	**Chicken-and-egg**	Acche guess ko accha alignment chahiye aur ulta; phir bhi dono saath sudharte hain.	`align.py circular`
69	**External aligner**	Alag tool (Montreal Forced Aligner) se durations. FastSpeech 2 ka tareeka.	theory
70	**Attention alignment**	Tacotron: attention khud seekhta hai kahan dekhna hai; galti pe shabd chhoot-te / dohrate hain.	`model.py Tacotron`
71	**Forward-sum loss**	Alignment ko CTC jaise loss se seekhna (FastPitch ka naya aligner).	theory

E. Acoustic model (lesson 5: model.py)

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
72	**Acoustic model**	Text se mel spectrogram banane wala network.	`model.py class Acoustic`
73	**Two-stage TTS**	Pehle text → mel, phir mel → waveform.	`speak.py acoustic model -> mel -> vocoder`
74	**Non-autoregressive TTS**	Saare frames ek saath, pichhle frame pe nirbhar nahi.	`model.py NON-AUTOREGRESSIVE`
75	**FastSpeech family**	Duration predictor + length regulator wala design.	`model.py FastSpeech`
76	**Text encoder**	Har akshar ko uske padosiyon ki jaankari ke saath vector banana.	`model.py def encode`
77	**Language embedding**	Bhasha ka vector, har akshar mein joda jaata hai.	`model.py self.langs`
78	**Duration predictor**	Chhota network: har akshar ke liye log(frames).	`model.py class DurationPredictor`
79	**Log-duration**	Duration ka log predict karna: 2 ko 4 kehna utna hi bura jitna 20 ko 40.	`model.py .log()`
80	**Length regulator**	Har akshar ka vector utni baar dohrana jitni uski duration. Text-time se audio-time.	`model.py def length_regulate`
81	**Upsampling by repetition**	Wahi kaam, PyTorch ki ek line mein.	`model.py torch.repeat_interleave`
82	**Mel decoder**	Dohraye gaye vectors ko chikna spectrogram banana.	`model.py def decode`
83	**Mel head**	Aakhri linear layer: d se 80 bands.	`model.py self.to_mel`
84	**Prior (mu)**	Encoder ka har akshar ke liye ek mel frame ka mota andaza. MAS isi pe chalta hai.	`model.py self.prior`
85	**Prior loss**	Andaze ko asli mel ke kareeb laana, alignment ke saath.	`model.py prior_loss`
86	**Mel loss (L1)**	Bane aur asli spectrogram ka fark.	`model.py mel_loss`
87	**Duration loss**	Predictor ko MAS ki durations sikhana.	`model.py dur_loss`
88	**Stop-gradient**	Duration loss encoder ko na mode, sirf predictor ko sikhaye.	`model.py h.detach()`
89	**Multi-task loss**	Teen losses ka jod ek saath kam karna.	`model.py mel_loss + prior_loss + dur_loss`
90	**Speed control**	Durations ko ek number se bhaag do: tez ya dheema, sur wahi.	`model.py speed`
91	**Inference path**	Training mein MAS ki durations, inference mein predictor ki.	`model.py def synthesize`
92	**Global mel normalization**	Poore mel ka ek mean aur ek std: bands ka aapsi fark bana rahe.	`model.py self.register_buffer("mean"`

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
93	**Over-smoothing**	L1 loss "ausat" spectrogram ko inaam deta hai, jo dhundhla hota hai.	`train.py blurred`
94	**Length ratio**	Dev pe predicted kul lambai / asli lambai. 1.00 = bolne ki raftaar sahi.	`train.py length`
95	**Listening sample**	Har epoch ke baad ek vaakya Griffin-Lim se bana ke sunna.	`train.py sample_`
96	**Pitch predictor**	FastPitch / FastSpeech 2: har akshar ka sur bhi predict karna. Hamare paas nahi.	theory
97	**Energy predictor**	Har frame ki loudness predict karna.	theory
98	**Variance adaptor**	Duration + pitch + energy predictors ka samooh.	theory
99	**One-to-many problem**	Ek vaakya kai tarah se bola ja sakta hai; model ko ek chunna hai.	theory
100	**Flow matching**	Matcha-TTS / Supertonic: shor se mel tak seedha raasta seekhna. Over-smoothing ka ilaaj.	theory
101	**Diffusion TTS**	Shor ko kai kadmon mein saaf karke spectrogram banana.	theory
102	**VITS**	Acoustic model aur vocoder ek saath (end-to-end). Piper yahi hai.	theory
103	**Codec language model**	Audio ko tokens bana ke LLM jaise predict karna (VALL-E, Orpheus).	theory

F. Vocoder (lesson 7: vocoder.py)

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
104	**Vocoder**	Mel spectrogram se waveform banane wala model.	`vocoder.py class Vocoder`
105	**Neural vocoder**	Phase ko seekh ke banana, Griffin-Lim ki tarah andaze se nahi.	`vocoder.py LEARNS`
106	**HiFi-GAN**	Mel ko 256 guna upsample karke waveform. Standard, par bhaari.	`vocoder.py HiFi-GAN`
107	**Vocos / iSTFT vocoder**	Frame rate pe hi raho, magnitude + phase predict karo, iSTFT baaki kare.	`vocoder.py Vocos`
108	**ConvNeXt block**	Depthwise conv + chhota MLP + residual. Attention nahi.	`vocoder.py class ConvNeXtBlock`
109	**Layer scale**	Har block shuru mein bahut chhota sudhaar jodta hai.	`vocoder.py self.scale`
110	**Magnitude + phase head**	Har frame ke liye 513 log-magnitudes aur 513 phases.	`vocoder.py N_FFT + 2`
111	**Phase wrapping**	Phase ek kon hai: 0 aur 2π ek hi. cos / sin se ye apne aap sambhalta hai.	`vocoder.py torch.complex`

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
112	**Magnitude clamp**	Shuru ki ek paagal prediction iSTFT ko uda na de.	`vocoder.py clamp(max=100.0)`
113	**Spectral loss**	Waveform nahi, spectrograms ki tulna. Alag phase wali do awaazein ek jaisi sun sakti hain.	`vocoder.py def spectral_loss`
114	**Multi-resolution STFT loss**	Teen window sizes: chhoti timing dekhti hai, lambi pitch.	`vocoder.py (512, 1024, 2048)`
115	**Spectral convergence**	Zor wale hisson ka relative fark.	`vocoder.py convergence`
116	**Log-magnitude loss**	Dheeme hisson ko bhi barabar wazan.	`vocoder.py log_magnitude`
117	**Random crops**	Vocoder ko poore vaakya nahi chahiye; chhote random tukde kaafi.	`vocoder.py def batch_from`
118	**GAN**	Do networks: ek banata hai, doosra asli-nakli pehchanta hai. Aakhri "buzz" isi se jaati hai.	`vocoder.py GAN`
119	**Discriminator (MPD / MRD)**	GAN ka jaanchne wala: alag periods / resolutions pe dekhta hai.	`vocoder.py class Discriminators`
120	**Feature matching loss**	Generator ko discriminator ki beech ki layers ke features milane ko kehna.	`vocoder.py def fooling_loss`
121	**Mismatch: GT vs predicted mel**	Vocoder asli mel pe train hota hai, par use acoustic model ka mel milta hai.	theory

G. Inference aur evaluation (lesson 8: speak.py)

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
122	**Real-time factor**	Compute ke second / audio ke second. TTS ke liye 0.1 se kam aam hai.	`speak.py real-time factor`
123	**Vocoder ablation**	Wahi mel, Griffin-Lim banam neural vocoder: fark sunna.	`speak.py --griffin-lim`
124	**MOS**	Mean opinion score: log 1-5 mein rate karte hain. TTS ka asli paimana.	theory
125	**Intelligibility (ASR round-trip)**	Banayi awaaz ko ASR se likhwa ke CER dekhna: shabd samajh aa rahe hain?	theory
126	**UTMOS**	Model jo MOS ka andaza lagata hai, insaanon ke bina.	theory
127	**Streaming TTS**	Vaakya-dar-vaakya (ya chunk) audio dena, poore jawab ka intezaar nahi.	theory
128	**Time to first audio**	Text milne se pehla sample bajne tak. Voice agent ke liye sabse zaroori.	theory

H. Expressive awaaz (lesson 10: model.py, vocoder.py, data.py)

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
129	**Over-smoothing**	L1/L2 loss saare sambhav tareekon ka AVERAGE sikhata hai: dhundhla mel, flat sur = robotic. Humne naapa: 17% kam frame-to-frame badlaav.	`model.py Lesson 10`
130	**Flow matching**	Noise se speech tak seedhi line pe chalna sikhana: har point pe "kis taraf jaana hai" (velocity).	`model.py def velocity`
131	**OT-CFM**	Optimal-transport conditional flow matching: x_t = (1-(1-s)t)x0 + t·x1, target u = x1-(1-s)x0. Matcha-TTS ka loss.	`model.py u = target - (1 - self.cfg.sigma_min) * x0`
132	**sigma_min**	t=1 pe data ke aas-paas chhoda gaya bahut chhota phailaav (1e-4).	`model.py sigma_min`
133	**Time embedding**	t (0 se 1) ko sinusoids + MLP se vector banana, taaki decoder jaane wo raaste mein kahan hai.	`model.py class TimeEmbedding`
134	**Euler steps (NFE)**	Noise se mel tak kitne kadam. Zyada = behtar par dheema. Humare CPU pe 4 steps: RTF 0.17.	`model.py x = x + v / steps`
135	**Temperature (sampling)**	Shuruaati noise ka size. Kam = shaant aur surakshit, zyada = zinda par galti ka khatra. Matcha: 0.667.	`model.py temperature`
136	**Speaker embedding**	Har awaaz ka ek vector; model ko batata hai kaun bol raha hai.	`model.py self.speakers`
137	**Style / emotion embedding**	Khushi, gussa, dukh... har style ka vector. Encoder (durations) aur decoder (sur) dono ko milta hai.	`model.py self.styles`
138	**Classifier-free guidance**	Training mein 20% baar style chhupao; bolte waqt v = v_none + w(v_style - v_none). w > 1 = emotion zyada.	`model.py guidance`
139	**Hinge loss (GAN)**	Critic ko asli pe +1 se upar, nakli pe -1 se neeche score dena sikhana. Vocos yahi use karta hai.	`vocoder.py def critic_loss`
140	**GAN fine-tuning**	Pehle se train vocoder ko critic ke saamne aage train karna; zero se GAN asthir hota hai.	`vocoder.py --gan`
141	**Expressive dataset**	Ek hi insaan alag emotions aur styles mein padhe. Rasa: Hindi aur Marathi, 2 awaazein, 6 emotions + baat-cheet, news, kahani.	`data.py def rasa`
142	**Gated dataset**	HF pe terms accept kiye bina download nahi hota (403).	`data.py Gated`
143	**Distillation ki seema**	Teacher (Supertonic) se seekha model teacher jaisa hi expressive ho sakta hai, zyada nahi. Isliye asli insaan ka data.	`data.py Real people`

Orchestration glossary

A. Real-time audio ki neenv

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
1	**Real-time audio**	Audio jo aate hi process hona chahiye; der hui to baat-cheet toot jaati hai.	`agent.py def run_mic`
2	**Audio frame**	Audio ka chhota tukda (20 ms) jo ek baar mein aage badhaya jaata hai.	`vad.py FRAME = 320`
3	**Frame size ka trade-off**	Chhota frame = kam delay par zyada overhead. 10-20 ms voice ka standard.	`agent.py blocksize=FRAME`
4	**Audio callback**	Sound driver apne thread se har naye frame pe ye function bulata hai.	`agent.py callback = lambda`
5	**Thread-safe handoff**	Driver ke thread se event loop tak data surakshit pahunchana.	`agent.py call_soon_threadsafe`
6	**Queue**	Frames ki line: producer (mic) aur consumer (pipeline) alag raftaar se chal sakte hain.	`agent.py asyncio.Queue()`
7	**Capture / playback**	Mic se lena, speaker pe bajana. Dono alag streams.	`agent.py sounddevice.InputStream`
8	**Non-blocking playback**	Audio bajana shuru karke turant lautna, taaki sunna chalu rahe.	`agent.py sounddevice.play`
9	**Decibel (dB)**	Loudness ka log scale: 10 log10(energy).	`vad.py 10 * math.log10`
10	**Buffer underrun**	Speaker ko samay pe audio na mile to "khat-khat" (glitch).	theory
11	**Jitter**	Packets ke aane ke samay ka upar-neeche hona.	theory
12	**Sample-rate mismatch**	STT 16 kHz chahta hai, TTS 22 kHz deta hai: har seema pe rate dekhna padta hai.	`agent.py need {SR}`

B. VAD (lesson 1: vad.py)

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
13	**VAD**	Voice activity detection: abhi koi bol raha hai ya nahi.	`vad.py class VAD`
14	**Energy VAD**	Frame ki loudness dekh ke faisla. Sabse purana tareeka.	`vad.py def is_voiced`
15	**Noise floor**	Kamre ki wo awaaz jab koi nahi bol raha.	`vad.py self.floor_db`
16	**Adaptive noise floor**	Floor khud seekhta hai: neeche tezi se, upar dheere.	`vad.py rate = 0.2 if level`
17	**Margin / threshold**	Floor se kitna upar ho to "bolna" maanein.	`vad.py margin_db`
18	**Hysteresis**	Shuru hone aur rukne ke alag niyam, taaki state baar-baar na palte.	`vad.py HYSTERESIS`
19	**Start debounce**	Itne ms lagataar awaaz ho tabhi "shuru". Ek click se trigger nahi.	`vad.py start_ms`

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
20	**Hangover / end silence**	Itne ms chuppi ho tabhi "khatm". Shabdon ke beech ke gap se nahi.	`vad.py end_ms`
21	**Pre-roll buffer**	Pichhle ~300 ms sambhal ke rakhna, kyunki "shuru" der se pata chalta hai.	`vad.py self.pre_roll`
22	**VAD event**	State badalne ki khabar: start ya end (end ke saath poora utterance).	`vad.py class Event`
23	**Utterance**	Do chuppiyon ke beech bola gaya ek tukda.	`vad.py self.utterance`
24	**Streaming VAD**	Ek-ek frame pe chalta hai, poori recording ka intezaar nahi.	`vad.py def feed`
25	**Run counter**	Lagataar kitne frames state ke khilaaf gaye. Hysteresis isi se banta hai.	`vad.py self.run`
26	**Energy VAD ki seema**	Awaaz aur kisi bhi zor ke shor mein fark nahi kar sakta.	`vad.py cannot do`
27	**Neural VAD (Silero)**	Chhota network jo speech ko shor se alag pehchanta hai. Asli agent yahi use karta hai.	`vad.py Silero`
28	**VAD ke false positive / negative**	Shor ko bolna samajhna / dheemi awaaz chhod dena.	theory
29	**Speech probability**	Neural VAD har frame pe 0-1 ka number deta hai; us pe threshold lagta hai.	theory

C. Turn-taking (lesson 2: turn.py)

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
30	**Turn**	Ek vyakti ki ek baari. Baat-cheet baariyon ka aana-jaana hai.	`pipeline.py class Turn`
31	**End-of-turn (EOT) detection**	Ye tay karna ki saamne wala bol chuka.	`turn.py class Endpointer`
32	**Endpointing**	Wahi, STT ki bhasha mein: utterance ka ant dhoondhna.	`turn.py def delay`
33	**Pause vs end**	Vaakya ke beech ka rukna aur baari ka ant: dono chuppi hain.	`turn.py MIDDLE of a turn`
34	**False cutoff**	Agent ne beech mein hi jawab de diya. Sabse buri galti.	`turn.py def false_cutoffs`
35	**Acoustic signal**	Kitni der se chuppi hai.	`turn.py acoustic`
36	**Semantic signal**	Jo kaha gaya wo poora lagta hai ya adhoora.	`turn.py semantic`
37	**LM-based turn detector**	Language model se poochna: kya agla token "vaakya khatm" hai?	`turn.py def lm_completeness`
38	**End tokens**	Newline aur . ? !: inki probability ka jod = "poora hone" ki sambhavna.	`turn.py enders`
39	**Min endpointing delay**	Poora lagne wale vaakya ke baad kitni chuppi ka intezaar.	`turn.py min_delay`

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
40	**Max endpointing delay**	Adhoora lagne wale vaakya ke baad.	`turn.py max_delay`
41	**Threshold ka matlab**	Ye "jawab dein ya nahi" tay nahi karta, sirf kitna sabr rakhna hai.	`turn.py threshold`
42	**Latency vs cutoff trade-off**	Jaldi jawab = zyada false cutoffs. Dono saath kam karna hi kala hai.	`turn.py latency budget`
43	**eot-bench**	LiveKit ka benchmark: 300 / 600 ms budget pe false cutoffs.	`turn.py eot-bench`
44	**Audio-based turn model**	Smart Turn jaisa model jo awaaz ke sur (prosody) se ant pehchanta hai.	theory
45	**Prosody**	Sur, lay, zor. "theek hai?" aur "theek hai." ka fark isi mein hai.	theory
46	**Backchannel**	"hmm", "haan" jaise shabd jo baari nahi lete. Inpe rukna nahi chahiye.	theory
47	**Adaptive endpointing**	Is user ke pichhle pauses dekh ke delay badalna.	theory

D. Pipeline aur streaming (lessons 3-4: models.py, pipeline.py)

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
48	**Cascaded pipeline**	STT → LLM → TTS, teen alag models ek ke baad ek.	`pipeline.py class Agent`
49	**Plugin interface**	Har model ek saade function ke peeche. Model badlo, pipeline wahi.	`models.py plugins`
50	**STT adapter**	Scratch recognizer ko `samples -> text` function banana.	`models.py def load_stt`
51	**TTS adapter**	`sentence -> (samples, rate)`.	`models.py def load_tts`
52	**LLM adapter**	`text -> pieces ka iterator`.	`models.py def load_llm`
53	**Model swap**	Wahi interface, doosra LLM (Gemma). Baaki code nahi badla.	`models.py def gemma`
54	**Token streaming**	LLM ke tokens bante hi aage bhejna, poore jawab ka intezaar nahi.	`models.py generate.stream`
55	**SSE (server-sent events)**	HTTP pe streaming: har line `data: {...}`, aakhir mein `[DONE]`.	`models.py data: [DONE]`
56	**Partial UTF-8**	Ek Devanagari akshar kai bytes ka hai; aadha aaye to rukna padta hai.	`models.py`
57	**Sentence chunking**	Token stream ko vaakyon mein baantna, har vaakya turant TTS ko.	`models.py def sentences`
58	**Minimum chunk**	"Hi." akela synthesize ho to kata-kata lagta hai; chhote tukde jodo.	`models.py min_chars`
59	**System prompt**	LLM ko diya gaya sthayi nirdesh: chhota bolo, markdown nahi.	`models.py system =`

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
60	**Script bridging**	STT Devanagari likhta hai, LLM Roman padhta hai: beech mein transliteration.	`models.py def romanize`
61	**Inherent vowel**	Har vyanjan ke saath chhupa "a"; shabd ke ant mein gir jaata hai.	`models.py carries an "a"`
62	**State machine**	Agent hamesha teen mein se ek state mein: LISTENING, THINKING, SPEAKING.	`pipeline.py self.state`
63	**Barge-in**	User beech mein bole to agent turant chup.	`pipeline.py def _interrupt`
64	**Playback cut**	Bajta hua audio usi pal rokna.	`pipeline.py self.stop()`
65	**Cancellation**	Tooti hui reply ka baaki kaam radd karna.	`pipeline.py self._task.cancel()`
66	**Generation counter**	Har reply ka number; purana number wala kaam khud ruk jaata hai.	`pipeline.py _generation`
67	**Stale result**	Thread ko maara nahi ja sakta; uska der se aaya nateeja phenk dete hain.	`pipeline.py generation != self._generation`
68	**Event loop**	Ek thread jo chhote kaamon ko baari-baari chalata hai.	`pipeline.py asyncio.create_task`
69	**Blocking call in a thread**	Model ka bhaari kaam worker thread mein, taaki loop mic sunta rahe.	`pipeline.py asyncio.to_thread`
70	**Async iterator bridge**	Blocking generator ko event loop se ek-ek kadam chalana.	`pipeline.py async def _aiter`
71	**Utterance merging**	User ruk ke phir bola: dono tukde ek hi baari mein.	`pipeline.py _pending`
72	**Playback clock**	Pichhla vaakya kab tak bajega, taaki agla uske baad shuru ho.	`pipeline.py _speaking_until`
73	**Event hook**	Pipeline ke andar kya hua, bahar batane ka raasta (log, UI).	`pipeline.py on_event`
74	**Interruption ke baad context**	Asli agents yaad rakhte hain ki user ne kitna jawab suna tha. Hamara nahi rakhta.	theory
75	**Chat history**	Pichhli baariyan LLM ko dobara dena. Hamara scratch LLM ek baari ka hai.	theory
76	**Tool / function calling**	LLM ka bahar ke function bulana (order status, calendar).	theory
77	**Filler / thinking sound**	Jawab mein der ho to "hmm" bajana.	theory
78	**Speech-to-speech model**	Ek hi model audio leke audio deta hai (Moshi, GPT realtime). Pipeline nahi.	theory
79	**Full-duplex**	Sunna aur bolna ek saath. Cascaded pipeline ise barge-in se nakal karta hai.	theory

E. Latency aur naap-tol (pipeline.py, agent.py, check.py)

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
80	**Time to first audio**	User ruka → agent ne bolna shuru kiya. Yahi mehsoos hota hai.	`pipeline.py first_audio_ms`
81	**Endpointing delay**	VAD hangover + endpointer ka intezaar. Aksar sabse bada hissa.	`pipeline.py endpoint_ms`
82	**STT latency**	Utterance → text.	`pipeline.py stt_ms`
83	**TTFT**	Time to first token: LLM ka pehla tukda.	`pipeline.py llm_first_ms`
84	**TTS first-chunk latency**	Pehle vaakya ka audio banne ka samay.	`pipeline.py tts_first_ms`
85	**Latency budget**	Kul delay ko stages mein baantna; ~800 ms ke andar baat-cheet sahaj lagti hai.	`agent.py def report`
86	**Streaming ka fayda**	Pehla audio ek vaakya ke baad, poore jawab ke baad nahi.	`check.py Streaming`
87	**Fake models**	Models ki jagah tay delay wale nakli functions: logic akela test hota hai.	`check.py def make`
88	**Scenario test**	Poori baat-cheet ka drishya chalake jaanchna: barge-in, resume.	`check.py scenario_barge_in`
89	**File-driven simulation**	Mic ki jagah recording ko real-time raftaar se pipeline mein daalna.	`agent.py def run_file`
90	**Echo test**	LLM ki jagah "jo suna wahi bolo": STT aur TTS ka akela test.	`agent.py "echo"`
91	**Real-time pacing**	Simulation mein bhi 20 ms ka frame 20 ms mein dena, kyunki pipeline ghadi dekhta hai.	`agent.py FRAME / SR`
92	**P50 / P95 latency**	Ausat nahi, percentile dekho: 5% baariyan kitni dheemi hain.	theory
93	**Observability**	Har turn ke stage-wise numbers production mein log karna.	theory

F. WebRTC aur transport (theory)

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
94	**Echo**	Speaker ki awaaz mic mein wapas aana: agent khud ko sun leta hai.	`agent.py hear ITSELF`
95	**AEC**	Acoustic echo cancellation: jo baja rahe ho use mic se ghata dena.	`agent.py echoCancellation`
96	**WebRTC**	Browser ka real-time audio/video standard: kam delay, UDP pe.	theory
97	**UDP vs TCP**	UDP khoya packet dobara nahi bhejta: real-time ke liye der se behtar hai na aana.	theory
98	**Opus**	WebRTC ka audio codec: 20 ms frames, 6-510 kbps.	theory

#	TERM	SEEDHI BHASHA MEIN	CODE MEIN KAHAN
99	**Jitter buffer**	Packets ko thoda rok ke sahi kram aur samay pe bajana.	theory
100	**Packet loss concealment**	Khoye packet ki jagah andaza bharna.	theory
101	**ICE**	Do peers ke beech raasta dhoondhne ki prakriya.	theory
102	**STUN**	Server jo batata hai tumhara public IP / port kya hai.	theory
103	**TURN**	Seedha raasta na mile to beech ka relay server.	theory
104	**NAT traversal**	Ghar ke router ke peeche se bahar wale se judna. ICE + STUN + TURN.	theory
105	**SDP offer / answer**	Dono taraf ka "main kya bhej / le sakta hoon" ka parcha.	theory
106	**Signaling**	SDP aur ICE candidates ka aadan-pradan. WebRTC ise tay nahi karta; aksar WebSocket.	theory
107	**DTLS**	UDP ke upar encryption ka handshake.	theory
108	**SRTP**	Encrypted audio/video packets.	theory
109	**SFU**	Selective forwarding unit: server jo har participant ka stream doosron ko aage bhejta hai. LiveKit yahi hai.	theory
110	**Room / participant / track**	LiveKit ke shabd: kamra, usme log (agent bhi ek participant), unke audio/video streams.	theory
111	**Agent worker / dispatch**	Process jo naye room ke liye agent ka job uthata hai.	theory
112	**WebSocket**	TCP pe do-tarfa connection. Simple par packet loss pe atakta hai (head-of-line blocking).	theory
113	**AGC / noise suppression**	Awaaz ka level barabar karna, background shor hatana. Browser ye bhi deta hai.	theory
114	**Telephony (SIP, 8 kHz)**	Phone calls ka raasta; audio 8 kHz, isliye phone pe STT mushkil.	theory
115	**DTMF**	Phone ke button ki tones.	theory